

MANUAL DAS PEÇAS

Manual de script Unity / C# — cada peça das aulas, aprofundada

Cada peça que aparece nas aulas ganha uma entrada aqui, no formato **o que é → pra que serve → regras e pegadinhas → onde tá no meu código → exemplos e formas de usar**. Bateu dúvida em casa? Procura aqui antes de procurar em outro lugar.

Nesta edição: cada peça ganhou também uma caixa **Simples assim** — a mesma ideia explicada como se fosse a primeira vez, com exemplo do dia a dia — e entraram as peças da aula de 04/09/2026 (ground check com `OverlapCircle`, limpeza do código, animação de pulo e queda, atrito zero e pivô do sprite).

(alimentado pelo Claude a cada peça ensinada — se faltar alguma, cobrar)

Sumário

Como usar este manual	4
PARTE 1 — FUNDAMENTOS	7
1.1 O esqueleto de todo script — MonoBehaviour e Update	7
1.2 Time.deltaTime — o relógio do quadro [peça futura, mas indispensável]	9
1.3 Campos public/private e o Inspector	11
1.4 Tipos de dado — o que cada caixa aceita guardar [peça de apoio]	12
1.5 Operadores, condições e laços — o C# que aparece dentro dos métodos [peça de apoio]	14
1.6 Métodos — funções minhas x alarmes da Unity [peça de apoio]	16
1.7 Rigidbody2D e rb.linearVelocity — o motor físico	18
1.8 transform — posição, escala, rotação, pai e filho	20
1.9 Prefab — o molde azul (a peça 2, feita no editor)	22
1.10 Debug.Log("...") — a prova no Console	23
1.11 Limpeza e estruturação do código — uma caixa, um dono; declarar uma vez só [peça em uso, aula de 04/09]	24
PARTE 2 — INPUT: CAMPAINHAS, RÉGUAS E TECLAS CRUAS	27
2.1 Input.GetButtonDown("Jump") — a campainha de APELIDO	27
2.2 A lista de apelidos de fábrica (Input Manager)	28
2.3.GetAxis x GetAxisRaw — a régua com e sem amortecimento [peça de apoio]	29
2.4 Input.GetKey e a tabela de KeyCode [peça de apoio]	29
2.5 Mouse — clique e posição [peça futura]	30
2.6 Criar um apelido novo no Input Manager — passo a passo do "Dash" [peça futura]	31
2.7 Aviso de fronteira: o "novo Input System" [só pra reconhecer]	31
PARTE 3 — FÍSICA 2D: COLLIDERS, TROMBADAS E TOQUES FANTASMAS	33
3.1 Colliders — a casca que a física enxerga [peça de apoio]	33
3.2 OnCollisionEnter2D(Collision2D colisao) — o alarme de trombada	34
3.3 Tags e Layers — etiquetas e camadas [peça de apoio]	36
3.4 OnTriggerEnter2D(Collider2D outro) — o toque fantasma [peça 4 do roadmap]	37
3.5 Ground check profissional — Physics2D.OverlapCircle e Raycast [peça em uso, aula de 04/09]	38
PARTE 4 — MOVIMENTO DO PERSONAGEM, PEÇA POR PEÇA	41
4.1 rb.linearVelocity também se LÊ — pulo com altura controlada	41
4.2 Eixos: x é pro LADO, y é pra CIMA	41
4.3 O pulo completo — a versão de referência [peça consolidada]	42
4.4 Coyote time e jump buffer — o pulo que perdoa [peça futura]	43
4.5 Dash de verdade — velocidade + tempo [peça futura]	44
4.6 Agachar — localScale hoje, collider amanhã [peça em uso + futura]	45
4.7 Virar o boneco — direcao + localScale x flipX [peça em uso]	47
4.8 Movimento.cs de referência — tudo junto [código de referência, não o meu arquivo]	47
PARTE 5 — FASE 2: COMBATE EM TEMPO REAL	50
5.1 public GameObject — caixinha de Inspector que guarda um OBJETO inteiro	50
5.2 Instantiate(molde, onde, rotação) — a máquina de carimbar	50
5.3 Dois scripts conversando — caixinha de vizinho + porta public	51
5.4 GetComponent<T>() — achar um componente sem arrastar [peça futura, chave]	52
5.5 Destroy — a borracha do carimbo [peça 4 do roadmap]	53
5.6 Espada.cs — o golpe que machuca [peça 4, código de referência]	54
5.7 Inimigo.cs — vida, dano e morte [peça 5, código de referência]	55
5.8 Ataque.cs completo — ouvido, cooldown e carimbo com direção [código de referência]	57
5.9 Vida do player, dano de contato e recomeço [peça futura]	58
5.10 Roadmap da Fase 2 (combate em tempo real, decidido 03/09)	59

PARTE 6 — TEMPO, CORROTINAS E ESTADOS	60
6.1 Timers com float e deltaTime — o padrão que já uso [peça em uso]	60
6.2 Invoke — agendar um método pelo nome [peça futura]	61
6.3 Corrotinas — IEnumerator e yield [peça futura]	61
6.4 enum + switch — a máquina de estados simples [peça futura]	63
6.5 static e o GameManager — uma caixa só pro jogo inteiro [peça futura]	64
6.6 Listas e arrays — várias caixas do mesmo tipo [peça futura]	65
PARTE 7 — PEÇAS DE APOIO: CÂMERA, CENAS, UI, SOM, ANIMAÇÃO	66
7.1 Câmera que segue o boneco [peça futura]	66
7.2 Cenas — SceneManager [peça futura]	66
7.3 UI rápida — texto na tela com TextMeshPro [peça futura]	67
7.4 Som — AudioSource [peça futura]	67
7.5 SpriteRenderer — o desenho [peça de apoio]	68
7.6 Animator — parâmetros e transições [peça em uso, aula de 04/09: IsJump e IsFall]	68
PARTE 8 — DICIONÁRIO DE ERROS	71
8.1 Erros de compilação (gritam vermelho no Console, o jogo nem dá Play)	71
8.2 Erros em tempo de execução (só aparecem com o jogo RODANDO)	73
8.3 Bugs silenciosos (sem mensagem nenhuma — o mais perigoso)	74
8.4 Checklist: "meu boneco não anda"	76
PARTE 9 — FERRAMENTAS: VS CODE, EDITOR DA UNITY, GIT	77
9.1 O ponto que aceita sugestão sozinho	77
9.2 IntelliSense de verdade (em casa) e o stub (no trabalho)	77
9.3 Atalhos do VS Code que pagam a conta	78
9.4 Editor da Unity — atalhos e hábitos	78
9.5 Git com Unity — o que viaja e o que não viaja	79
APÊNDICES	80
A. Cheat sheet — as linhas que mais uso	80
B. Template de script novo	81
C. Receita de objeto novo (o que pendurar em quê)	81
D. Glossário rápido	82

Como usar este manual

- **As entradas originais das aulas estão todas aqui**, com o texto de sempre. O que foi acrescentado vem marcado como *Mais fundo*, *Exemplos* ou *Formas de usar*, e as peças novas (ainda não vistas em aula) vêm com a etiqueta **[peça futura]** — são as próximas que vão aparecer.
- **Nesta edição entrou uma camada a mais:** a caixa amarela **Simples assim**, no começo de cada peça, explica a mesma coisa como se fosse a primeira vez (uma comparação do dia a dia + o exemplo mínimo). Quem já entendeu pula; quem travou começa por ela. As caixas verdes **Passo a passo** dizem onde clicar no editor, e as peças da aula de 04/09/2026 vêm marcadas **[aula de 04/09]** — ground check com círculo (3.5), faxina do código (1.11), animação de pulo/queda (7.6), fricção zero (3.1) e pivô do sprite (4.6).
- **"Minha linha X"** sempre se refere a `aula_8/Assets/Scripts/Movimento.cs` (e `ataques/Ataque.cs` na Fase 2) do jeito que estavam em 04/09/2026, depois da faxina do ground check. Quando eu editar o arquivo, o número muda; o que vale é a **receita**, não o número.
- Código de exemplo do manual é código de **referência**: compila e roda, mas não é o meu arquivo. Copiar pedaço, nunca o arquivo inteiro por cima do meu.
- **Ordem de leitura sugerida quando algo quebrou:** Parte 8 (dicionário de erros) → a peça específica → Parte 9 (ferramentas).

Legenda das metáforas do manual

apelido	o que é de verdade	exemplo
coleira	o "dono." antes do membro — a caixa mora dentro de alguém	<code>rb.linearVelocity,</code> <code>transform.localScale</code>
caixinha	campo <code>public</code> (ou <code>[SerializeField]</code>) visível no Inspector	<code>public float velocidade</code>
gaveta	campo <code>private</code> : só o próprio script enxerga	<code>private bool estaNoChao</code>
porta	campo <code>public</code> : vizinho pode ler/escrever	<code>public float direcao</code>
carimbo	<code>new Vector2(...)</code> , <code>Instantiate(...)</code> — fabrica uma cópia nova	<code>new Vector2(x, y)</code>
campainha	botão sim/não do Input	<code>Input.GetButtonDown("Jump")</code>
régua	eixo que devolve um número	<code>Input.GetAxisRaw("Horizontal")</code>
senha exata	string ou nome que o compilador NÃO confere	<code>"Jump"</code> , <code>OnCollisionEnter2D</code>
alarme	método de evento que a Unity chama sozinha	<code>OnCollisionEnter2D</code> , <code>Update</code>
boletim de ocorrência	o parâmetro que a Unity preenche e entrega no alarme	<code>Collision2D colisao</code>
as 3 irmãs	as três receitas de <code>Vector2</code> que preservam o eixo alheio	andar / pular / cortar
furo	bug conhecido, deixado de propósito pra testar depois	pulo no ar ao cair da beirada (fechado em 04/09 — 3.5)
checador	objeto vazio filho, na sola do pé, onde o círculo pergunta "tem chão?"	<code>checadorDeChao</code> (3.5)

apelido	o que é de verdade	exemplo
sombra	variável local com o nome de um campo — esconde a caixa de verdade dentro do método	<code>bool estaNoChao = true;</code> dentro de método (1.11)
interruptor	parâmetro <code>bool</code> do Animator que o script liga/desliga	<code>animator.SetBool("IsJump", true)</code> (7.6)

Os 3 tipos de erro (decorar — aparece o manual inteiro)

tipo	quando aparece	exemplo	como se acha
compilação	na hora de salvar/buildar, grita vermelho no Console	<code>CS0103</code> , <code>CS1955</code> , <code>CS1002</code>	Console → duplo-clique pula pra linha
silencioso	nunca aparece — a Unity só não chama	<code>void update()</code> , <code>OnCollisionEnter</code> sem 2D	comparar letra por letra com a senha; <code>Debug.Log</code> dentro pra provar que roda
de STRING	só explode com o jogo RODANDO	<code>GetButtonDown("fire1")</code> , <code>CompareTag("ground")</code>	Console em Play; conferir maiúsculas nas tabelas do manual
de referência (runtime)	caixinha vazia, objeto destruído	<code>NullReferenceException</code>	Console aponta a linha; olhar o Inspector

Se estou começando do zero: as 12 ideias que o manual inteiro usa

Tudo neste manual se apoia nestas ideias. Se uma peça não fizer sentido, é bem provável que uma linha desta tabela esteja faltando na cabeça.

ideia	em uma frase	onde aprofundar
GameObject	qualquer "coisa" da cena: o boneco, o chão, a câmera. Sozinho é só um nome com posição	D
componente	uma peça pendurada no GameObject que dá uma habilidade: desenhar (<code>SpriteRenderer</code>), ter peso (<code>Rigidbody2D</code>), ter casca (<code>Collider</code>), pensar (o meu script)	1.1
script / classe	um arquivo <code>.cs</code> com uma classe do mesmo nome; pendurado num objeto, vira componente	1.1
variável (caixa)	um nome que guarda um valor: <code>velocidade</code> , <code>estaNoChao</code> . Tem tipo (<code>float</code> , <code>bool</code>) e um dono só	1.3, 1.4, 1.11
método	uma receita com nome; escrevo uma vez e chamo com parênteses quantas vezes quiser	1.6
alarme (evento)	método que a UNITY chama sozinha na hora certa: <code>Update</code> todo quadro, <code>OnCollisionEnter2D</code> na trombada	1.1, 3.2
quadro (frame)	uma passada do jogo (~60 por segundo). O que está no <code>Update</code> acontece uma vez por quadro	1.1, 1.2
Inspector	o painel onde as caixas <code>public</code> aparecem pra ajustar e arrastar objetos — e onde o valor do Inspector vence o do código	1.3
Hierarchy / Project	Hierarchy = os objetos da cena (com pai e filho); Project = os arquivos (scripts, sprites, prefabs)	1.8, 1.9

ideia	em uma frase	onde aprofundar
Rigidbody2D + Collider	o peso e a casca: sem Rigidbody nada cai; sem Collider tudo atravessa	1.7, 3.1
Vector2	dois números juntos, x e y: velocidade, posição, escala. Se carimba inteiro com <code>new Vector2(x, y)</code>	1.7, 4.2
Console	onde a Unity grita os erros e onde o <code>Debug.Log</code> deixa bilhetes. Duplo-clique na linha abre o código	1.10, Parte 8

Lendo um script pela primeira vez, a ordem é sempre a mesma: **1)** os `using` no topo (a caixa de ferramentas), **2)** a classe (o nome do arquivo), **3)** as caixas `public/private` (o que o script guarda), **4)** os métodos (o que ele faz), **5)** dentro do `Update`, linha por linha, lendo em voz alta: "se ... então ...".

```
using UnityEngine;                                // 1) ferramentas

public class Movimento : MonoBehaviour              // 2) a classe = o arquivo Movimento.cs
{
    public float velocidade = 5f;                  // 3) caixa na vitrine (Inspector)
    public Rigidbody2D rb;                         // caixa que guarda um componente
    private bool estaNoChao;                       // caixa na gaveta

    void Update()                                  // 4) alarme: a Unity chama todo quadro
    {
        float lado = Input.GetAxisRaw("Horizontal"); // 5) "lê o teclado: -1, 0 ou +1"
        rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y); // "anda; mantém o y"
        if (Input.GetButtonDown("Jump") && estaNoChao) // "SE apertou pular E está
no chão"
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, 12f); // "ENTÃO joga pra cima"
    }
}
```

PARTE 1 — FUNDAMENTOS

(a Fase 1 inteira, registrada depois que o manual nasceu, e agora aprofundada)

1.1 O esqueleto de todo script — MonoBehaviour e Update

* Simples assim

Um script é uma folha de instruções. Sozinha, na pasta, ela não faz nada — precisa estar **pendurada** num objeto da cena, igual um adesivo colado no boneco. MonoBehaviour é o que transforma a folha em adesivo. Dentro dela, Update é a instrução que a Unity relê umas 60 vezes por segundo: "tem tecla apertada? então mexe". Eu não chamo o Update; eu só escrevo o que ele faz e a Unity liga pra ele o tempo inteiro.

```
void Update()
{
    Debug.Log("oi");    // aparece no Console ~60 vezes por segundo, sem eu pedir
}
```

- **O que é:** `public class X : MonoBehaviour` é o casco que transforma uma classe C# em COMPONENTE pendurável num GameObject. Script solto na pasta não roda: só executa **pendurado** num objeto da cena.
- `void Update()` : eu nunca chamo — **a Unity chama**, uma vez POR QUADRO (~60x/segundo). Tudo que é "escutar tecla / mexer todo instante" mora nele. Família de eventos: Update (todo quadro), OnCollisionEnter2D (na trombada) — a Unity liga pra eles, nunca o contrário.
- **Pegadinha:** o NOME do arquivo tem que ser igual ao da classe (`Movimento.cs` ↔ `class Movimento`), senão a Unity recusa pendurar.

Mais fundo: a anatomia de um script novo

```
using UnityEngine;                                // caixa de ferramentas da Unity: Vector2, Rigidbody2D, Input,
Debug...

public class Movimento : MonoBehaviour // nome do ARQUIVO = nome da CLASSE (Movimento.cs)
{
    // ---- caixinhas (aparecem no Inspector) ----
    public float velocidade = 5f;
    public Rigidbody2D rb;

    // ---- gavetas (só este script enxerga) ----
    private bool estaNoChao;

    void Start()    // a Unity chama UMA vez, no primeiro quadro em que o objeto existe
    {
        Debug.Log("nasci");
    }

    void Update()    // a Unity chama TODO quadro
    {
        // escutar teclado, mexer todo instante
    }
}
```

Tudo que é do script fica **dentro das chaves da classe**. Código fora delas → erro CS0116 ("um namespace não pode conter diretamente membros"). Método dentro de método → CS1513 / CS1519.

Mais fundo: a família completa dos alarmes (o ciclo de vida)

A Unity chama cada um destes em momentos fixos. Nomes são **senha exata** — errou uma letra, é erro silencioso (compila, nunca roda).

alarme	a Unity chama quando	frequência	uso típico
<code>Awake()</code>	o objeto é carregado na cena (antes de tudo)	1 vez	pegar meus próprios componentes (<code>GetComponent</code>)
<code>OnEnable()</code>	o componente/objeto é ligado	toda vez que liga	religar coisas depois de <code>SetActive(true)</code>
<code>Start()</code>	antes do 1º <code>Update</code> (só se o script está ligado)	1 vez	conversar com OUTROS objetos, valores iniciais
<code>FixedUpdate()</code>	a cada passo da física (0.02 s = 50×/s, fixo)	50×/s	aplicar força/velocidade no <code>Rigidbody</code>
<code>Update()</code>	todo quadro desenhado	~60×/s (varia)	ler teclado, timers, lógica
<code>LateUpdate()</code>	depois de TODOS os <code>Update</code> do quadro	~60×/s	câmera seguir o boneco (ele já andou)
<code>OnCollisionEnter2D/Stay2D/Exit2D</code>	collider sólido encostou / continua / soltou	por contato	chão, dano, caixa
<code>OnTriggerEnter2D/Stay2D/Exit2D</code>	collider fantasma (Is Trigger) atravessou	por contato	moeda, golpe, checkpoint
<code>OnDisable()</code>	o componente/objeto é desligado	toda vez que desliga	parar timers
<code>OnDestroy()</code>	o objeto vai ser destruído	1 vez	limpeza final

Ordem num objeto que acabou de nascer: `Awake` → `OnEnable` → `Start` → (`FixedUpdate`...) → `Update` → `LateUpdate`.

`Awake` × `Start` — a regra prática:

- `Awake`: "me arrumo" — só mexo em MIM (`rb = GetComponent<Rigidbody2D>()`);).
- `Start`: "falo com os outros" — todo mundo já passou pelo `Awake`, então as caixinhas dos vizinhos já estão preenchidas.
- Pegadinha: `Start` NÃO roda se o checkbox do script estiver desmarcado; `Awake` roda mesmo assim.

`Update` × `FixedUpdate` — onde botar o quê:

- Teclado sempre no `Update`. `GetButtonDown` só vale por UM quadro; no `FixedUpdate` (50×/s fixo) ele pode cair entre dois passos e o pulo "falha às vezes".
- Física (velocidade, força) idealmente no `FixedUpdate`. **Meu script hoje** crava `linearVelocity` dentro do `Update` e funciona — é aceitável num jogo simples. A mudança pro `FixedUpdate` é peça futura (receita em 1.7, "Formas de usar").
- Câmera no `LateUpdate`: ela mede a posição do boneco DEPOIS que ele já se mexeu no quadro (senão treme).

Regras e pegadinhas extras

- `void update()` (u minúsculo), `void Update(int x)` (com parâmetro), `void OnCollisionEnter2D()` (sem parâmetro) → **silencioso**: compila, nunca é chamado.
- Dois `void Update()` na mesma classe → CS0111 (membro duplicado). Um por script.

- Esqueceu `using UnityEngine;` → CS0246 ("tipo Vector2 não encontrado").
- Esqueceu `: MonoBehaviour` → a Unity recusa pendurar ("script class cannot be found" / não aparece na lista do Add Component).
- Script pendurado num objeto **desativado** (checkbox do topo do Inspector desmarcado) não roda nada. Checkbox **do componente** desmarcado: para o `Update`, `Start` e família — mas os alarmes de colisão/trigger ainda chegam. Pra calar tudo, desativar o `GameObject` ou o `collider`.
- Um script pode estar pendurado em VÁRIOS objetos: cada objeto tem sua cópia com suas caixinhas. Mudar a caixinha de um não muda a do outro.

Exemplos

```
// Contador de quadros — prova visual de que a Unity chama Update sem parar
private int quadros = 0;

void Update()
{
    quadros++;
    if (quadros % 60 == 0) // a cada ~1 segundo
        Debug.Log("já se passaram " + quadros + " quadros");
}
```

```
// Start roda uma vez: bom pra "pegar" o rb sem arrastar no Inspector
void Awake()
{
    rb = GetComponent<Rigidbody2D>(); // procura o Rigidbody2D NESTE objeto
}
```

1.2 Time.deltaTime — o relógio do quadro [peça futura, mas indispensável]

* Simples assim

Imagina dois PCs: um desenha 60 quadros por segundo, o outro 30. Se eu digo "anda 1 passo por quadro", no PC rápido o boneco anda o dobro. `Time.deltaTime` é quanto tempo aquele quadro demorou (um pedacinho de segundo). Multiplicando por ele, "por quadro" vira "por segundo" e o jogo fica igual em qualquer PC.

Regra de bolso: mexi na posição ou num timer com a mão → multiplico por `Time.deltaTime`; mexi em `rb.linearVelocity` → a física já cuida do tempo.

- **O que é:** quanto tempo (em segundos) passou desde o quadro anterior. A 60 fps vale ~0,016; a 30 fps vale ~0,033. Muda a cada quadro.
- **Pra que serve:** transformar "por quadro" em "por segundo". Qualquer coisa que EU mexo direto no `Update` (posição pelo `transform`, timers, contadores) tem que multiplicar por `Time.deltaTime`, senão o jogo roda mais rápido num PC mais rápido.
- **Regras / pegadinhas:**
 - `rb.linearVelocity` **já é por segundo** — o motor de física cuida do tempo. Meu `Movimento.cs` não precisa de `deltaTime` no andar nem no pulo.
 - `transform.Translate`, `transform.position += ...`, `tempo -= ...` → precisam.
 - Dentro do `FixedUpdate` a Unity entrega `Time.fixedDeltaTime` (0.02) automaticamente quando você usa `Time.deltaTime` — pode usar o mesmo nome.
 - `Time.timeScale = 0f` pausa o jogo: `deltaTime` vira 0 e todo timer congela. Menu de pausa que precisa de tempo usa `Time.unscaledDeltaTime`.

• A família do tempo:

membro	o que devolve
<code>Time.deltaTime</code>	segundos desde o quadro anterior
<code>Time.time</code>	segundos desde que o jogo começou (só cresce)
<code>Time.fixedDeltaTime</code>	passo da física (0.02 s por padrão)
<code>Time.timeScale</code>	velocidade do mundo: 1 normal, 0.5 câmera lenta, 0 pausa
<code>Time.unscaledDeltaTime</code>	<code>deltaTime</code> que ignora o <code>timeScale</code>
<code>Time.frameCount</code>	número do quadro atual

Exemplos

```
// Cooldown: um número que desce até zero
public float tempoRecarga = 0.5f;
private float recarga = 0f;

void Update()
{
    recarga -= Time.deltaTime; // desce em segundos, não em quadros
    if (Input.GetButtonDown("Fire1") && recarga <= 0f)
    {
        Atacar();
        recarga = tempoRecarga; // recarrega
    }
}
```

```
// Andar SEM física (ex.: plataforma que se move sozinha) – precisa do deltaTime
public float velocidade = 2f;
void Update()
{
    transform.Translate(Vector2.right * velocidade * Time.deltaTime); // 2 unidades por SEGUNDO
}
```

```
// Câmera lenta ao tomar dano (voltar pra 1f depois!)
Time.timeScale = 0.3f;
```

1.3 Campos public/private e o Inspector

* Simples assim

Campos são as caixas onde o script guarda coisas: números, o Rigidbody, o prefab. **public** = caixa na vitrine: aparece no Inspector e o vizinho pode mexer. **private** = caixa na gaveta: só este script vê. O detalhe que engana todo mundo: depois que a caixa **public** aparece no Inspector, **o número que vale é o do Inspector**, não o do código. Mudei no código e nada mudou? Olhar o Inspector.

```
public float velocidade = 5f; // vitrine: aparece no Inspector
private bool estaNoChao;     // gaveta: só eu vejo
```

- **public no topo da classe** = caixinha visível no Inspector (números ajustáveis com o jogo rodando, arrastões de objeto) E porta aberta pra outros scripts lerem. **private** = gaveta só do dono: nem Inspector, nem vizinho enxergam.
- **float pede o f** : 5f, 0.5f — sem o f o número é **double** e o compilador reclama da mistura.
- **PEGADINHA GRANDE:** depois que um campo **public** aparece no Inspector, o valor DO INSPECTOR vence o do código. Mudar `velocidade = 5f` pra `8f` no código NÃO muda o que o Inspector já gravou — ajustar lá, ou Reset no componente.

Mais fundo: as 4 combinações de caixinha e porta

declaração	aparece no Inspector?	vizinho lê?	quando usar
<code>public float x;</code>	sim	sim	o que uso hoje: rápido e simples
<code>[SerializeField] private float x;</code>	sim	não	o padrão profissional: ajusto no Inspector sem abrir a porta
<code>[HideInInspector] public float x;</code>	não	sim	porta pro vizinho sem poluir o Inspector
<code>private float x;</code>	não	não	estado interno (<code>estaNoChao</code> , timers)

`[SerializeField]` é o jeito de ter a caixinha SEM virar porta — quando eu quiser trancar a `direcao` de novo mas continuar vendo no Inspector, é ele.

Mais fundo: enfeites do Inspector (atributos entre colchetes)

```
[Header("Movimento")] // título em negrito no Inspector
[Tooltip("Unidades por segundo")] // texto ao passar o mouse
public float velocidade = 5f;

[Range(0f, 30f)] // vira um slider
public float forcaPulo = 12f;

[Space(10)] // respiro vertical
[Header("Referências")]
public Rigidbody2D rb;
```

Regras e pegadinhas extras

- **Mexeu no Inspector com o jogo RODANDO? Volta tudo no Stop.** O Play Mode é um rascunho: ajustou `forcaPulo` pra 14 e ficou bom → anota, ou no componente clica nos três pontinhos → *Copy Component* ainda em Play → dá Stop → três pontinhos → *Paste Component Values*.

- **Reset do componente:** três pontinhos (:) no cabeçalho do componente → *Reset*. Recarrega os valores escritos no código e limpa as caixinhas de objeto (vai ter que arrastar o `rb` de novo).
- **Renomeou o campo no código? O Inspector esquece o valor.** `velocidade` → `velocidadeAndar` volta pro valor do código e desarrasta referências. Renomear cedo, ou usar `[FormerlySerializedAs("velocidade")]` (`using UnityEngine.Serialization;`) em cima do campo novo.
- Caixa de OBJETO vazia (`rb`, `golpePrefab`, `movimento`) = `NullReferenceException` na primeira linha que usar. O Console aponta a linha; o conserto é arrastar.
- Campo `public` com valor inicial (`= 5f`) só usa esse valor na PRIMEIRA vez que o componente é pendurado. Depois, manda o Inspector.
- Tipos que viram caixinha: `int`, `float`, `bool`, `string`, `Vector2/3`, `Color`, `GameObject`, `Transform`, qualquer componente (`Rigidbody2D`), qualquer script meu (`Movimento`), `LayerMask`, `enum`, arrays e `List<>` desses tipos.

Exemplos

```
public class Movimento : MonoBehaviour
{
    [Header("Andar")]
    public float velocidade = 5f;

    [Header("Pular")]
    public float forcaPulo = 12f;
    [Range(0f, 1f)] public float corteDoPulo = 0.5f; // fração do corte – antes era número mágico

    [Header("Referências")]
    public Rigidbody2D rb;

    [SerializeField] private bool estaNoChao; // eu vejo no Inspector, vizinho não mexe
}
```

1.4 Tipos de dado — o que cada caixa aceita guardar [peça de apoio]

* Simples assim

Tipo é o formato da caixa: `int` guarda inteiro (3 vidas), `float` guarda número com vírgula (velocidade 5.5), `bool` guarda sim/não (`estaNoChao`), `string` guarda texto ("Jump"). O C# confere o formato e reclama se eu guardar 2.5 numa caixa de inteiro. O `f` no fim do `float` (`5f`) é só o jeito de avisar "esse número é float, não double".

```
int vidas = 3; // inteiro
float velocidade = 5f; // com vírgula (e com o f)
bool estaNoChao = true; // sim/não
string apelido = "Jump"; // texto entre aspas
```

- **O que é:** toda variável tem um TIPO, e o tipo diz o que cabe nela. O compilador confere: botar 2.5 numa caixa `int` é erro, botar texto numa caixa `float` é erro.
- **Os tipos que aparecem nos meus scripts:**

tipo	guarda	exemplo	observação
<code>int</code>	número inteiro	<code>int vida = 3;</code>	sem casa decimal; 7 / 2 dá 3

tipo	guarda	exemplo	observação
float	número com vírgula	float velocidade = 5f;	sempre o f
double	número com vírgula, mais preciso	5.0 (sem f)	a Unity não usa; é ele que causa o CS0029
bool	verdadeiro/falso	bool estaNoChao = true;	resposta de todo if
string	texto	string nome = "Boneco";	aspas duplas; "5" não é o número 5
Vector2	dois floats (x, y)	new Vector2(3f, 0f)	velocidade, direção 2D
Vector3	três floats (x, y, z)	new Vector3(1f, 1f, 1f)	posição, escala (z existe mesmo em 2D)
GameObject	um objeto inteiro da cena/prefab	public GameObject golpePrefab;	o "corpo"
Transform	a posição/escala/rotação de um objeto	public Transform pe;	todo GameObject tem um
Rigidbody2D, Collider2D, SpriteRenderer	componentes	public Rigidbody2D rb;	arrasta o objeto, a Unity acha o componente
Movimento, Inimigo	os MEUS scripts como tipo	public Movimento movimento;	peça 3.5
KeyCode	uma tecla	KeyCode.S	lista fixa (Parte 2)
Color	uma cor	Color.red, new Color(1f, 0f, 0f)	valores de 0 a 1, não 0 a 255
LayerMask	um conjunto de camadas	public LayerMask camadaChao;	caixinha vira lista de checkboxes

• Regras / pegadinhas:

- `int / int` é divisão INTEIRA: `1 / 2` dá 0, `7 / 2` dá 3. Quer decimal? `1f / 2f`.
- `float` → `int` não é automático: `(int)3.7f` dá 3 (corta), `Mathf.RoundToInt(3.7f)` dá 4.
- `int` → `float` é automático: `float x = vida;` funciona.
- `Vector2` ↔ `Vector3` se convertem sozinhos: `transform.position = new Vector2(1f, 2f);` funciona (z vira 0). `Vector2 v = transform.position;` funciona (joga o z fora).
- Caixa de objeto vazia guarda `null`. `if (rb == null) Debug.LogError("rb vazio!");` é um guarda útil.
- Nome de variável: começa com letra, sem espaço, sem acento por segurança (`forçaPulo`, não `força pulo`). Maiúscula importa: `velocidade` e `Velocidade` são duas caixas diferentes.
- `var x = 5f;` deixa o compilador adivinhar o tipo. Evitar por enquanto: esconde exatamente a informação que estou aprendendo a ler.

Exemplos

```
int   moedas    = 0;
float vida      = 10f;
bool  vivo      = true;
string nome     = "Boneco";

moedas += 1;           // 1
vida   -= 2.5f;        // 7.5
```

```
vivo    = vida > 0f;                                // true
Debug.Log(nome + " tem " + vida + " de vida"); // "Boneco tem 7.5 de vida"
Debug.Log($"{nome} tem {vida} de vida");         // mesma coisa, com $ e chaves
```

1.5 Operadores, condições e laços — o C# que aparece dentro dos métodos [peça de apoio]

* Simples assim

if é a pergunta: "se isso for verdade, faz aquilo". == pergunta se é igual; = guarda um valor (trocar os dois é o erro mais comum). && é "e", || é "ou", ! é "não". for repete um número de vezes; while repete enquanto algo for verdade (se nunca deixar de ser, a Unity trava).

```
if (Input.GetButtonDown("Jump") && estaNoChao) // lê-se: SE apertou pular E está no chão
    Pular();                                   // ENTÃO pula
```

Operadores

grupo	operadores	lembrete
conta	+ - * / %	% é o resto: 7 % 2 dá 1 (par/ímpar, "a cada N")
atalho	+= -= *= /= ++ --	vida -= 1; é vida = vida - 1;
comparação	== != > < >= <=	== compara , = atribui — if (vida = 0) nem compila
lógica	&& \ \ !	E, OU, NÃO — !estaNoChao lê-se "não está no chão"
ternário	cond ? a : b	if-else de uma linha (1.8)

if / else if / else

```
if (vida <= 0)
{
    Morrer();
}
else if (vida < 3)
{
    Debug.Log("cuidado");
}
else
{
    Debug.Log("tranquilo");
}
```

- **Chaves sempre**, mesmo pra uma linha. if sem chaves agarra SÓ a primeira linha seguinte — a segunda roda sempre, e o bug é invisível.
- if (cond); com ponto-e-vírgula no fim = if vazio; o bloco de baixo vira órfão e roda sempre (a "prima" que apareceu no dash v1).
- Comparar float com == é traiçoeiro (0.1f + 0.2f não é exatamente 0.3f). Usar > / < ou Mathf.Approximately(a, b).

switch — vários if numa lista

```
switch (estado)
{
    case Estado.Parado:    Debug.Log("parado"); break;
    case Estado.Andando:   Debug.Log("andando"); break;
    default:               Debug.Log("outro");   break;    // opcional
}
```

Cada `case` termina com `break`. Combina com `enum` (Parte 6.4).

Laços

```
// for: repetir N vezes (nascer 5 moedas em fila)
for (int i = 0; i < 5; i++)
{
    Instantiate(moedaPrefab, new Vector2(i * 1.5f, 0f), Quaternion.identity);
}

// foreach: passar por cada item de uma lista/array (Parte 6.6)
foreach (GameObject inimigo in inimigos)
{
    inimigo.GetComponent<Inimigo>().TomarDano(1);
}

// while: repetir ENQUANTO — cuidado
int tentativas = 0;
while (tentativas < 10)
{
    tentativas++;
}
```

- **PEGADINHA MORTAL do while** : um `while` cuja condição nunca vira falsa **congela a Unity inteira** (não é o jogo, é o editor — só matando no Gerenciador de Tarefas). Salvar a cena antes de dar Play sempre que houver `while` novo. Preferir `for`.
- `Update` já É um laço (um por quadro). Não precisa de `while (true)` pra "ficar escutando".

Mathf e Random — a calculadora

chamada	devolve	uso
<code>Mathf.Abs(x)</code>	x sem sinal	"está andando?" → <code>Mathf.Abs(lado) > 0.01f</code>
<code>Mathf.Sign(x)</code>	-1 ou +1	direção: <code>direcao = Mathf.Sign(lado)</code>
<code>Mathf.Clamp(x, min, max)</code>	x preso no intervalo	vida nunca passa de 10 nem cai de 0
<code>Mathf.Min(a, b)</code> / <code>Mathf.Max(a, b)</code>	o menor / o maior	limitar velocidade de queda
<code>Mathf.Lerp(a, b, t)</code>	ponto entre a e b (t de 0 a 1)	suavizar câmera
<code>Mathf.MoveTowards(a, b, passo)</code>	a andando em direção a b, no máximo <code>passo</code>	frear devagar
<code>Mathf.RoundToInt(x)</code> / <code>FloorToInt</code> / <code>CeilToInt</code>	arredonda	mostrar vida inteira na tela
<code>Mathf.Approximately(a, b)</code>	bool: são "iguais"?	comparar floats

chamada	devolve	uso
<code>Random.Range(0, 3)</code>	int 0, 1 ou 2 (o máximo NÃO entra)	sortear item de lista
<code>Random.Range(0f, 3f)</code>	float de 0 a 3 (o máximo entra)	posição aleatória
<code>Random.value</code>	float de 0 a 1	"10% de chance": <code>Random.value < 0.1f</code>

Pegadinha clássica: `Random.Range(1, 6)` sorteia de 1 a **5**. Pra dado de 6 faces é `Random.Range(1, 7)`.

1.6 Métodos — funções minhas x alarmes da Unity [peça de apoio]

* Simples assim

Método é uma receita com nome. Escrevo a receita uma vez (`void Pular() { ... }`) e depois só chamo pelo nome (`Pular();`) quantas vezes quiser. `void` quer dizer "não devolve nada"; um método `bool EstaVivo()` devolve uma resposta que cabe num `if`. Os parênteses na chamada são obrigatórios: `Pular` sem eles é só o nome da receita; `Pular()` é "executa".

- **O que é:** um método é um bloco de código com nome, que roda quando alguém CHAMA. `Update` e `OnCollisionEnter2D` são métodos que **a Unity** chama; `Pular()`, `TomarDano(1)` são métodos que **eu** chamo.

• A anatomia:

```
//  tipo de retorno  nome      parâmetros
//      |              |          |
//      void          Pular      ()
//      {
//          rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo);
//      }
//
//      void TomarDano(int dano)          // recebe um número
//      {
//          vida -= dano;
//      }
//
//      bool EstaVivo()                  // devolve um bool
//      {
//          return vida > 0;
//      }
//
//      float DistanciaAte(Transform alvo) // recebe um Transform, devolve float
//      {
//          return Vector2.Distance(transform.position, alvo.position);
//      }
```

- **Chamar:** `Pular();` — **parênteses obrigatórios**, mesmo sem parâmetro. `TomarDano(2);` passa o 2 pra dentro. `if (EstaVivo()) ...` usa o que voltou.
- `void` = "não devolve nada". Método com tipo (`bool`, `float`) **tem** que ter `return` em todo caminho, senão CS0161.
- **Pra que serve:** picar o `Update` em pedaços com nome. Um `Update` de 40 linhas vira:

```
void Update()
{
    Andar();
    Pular();
}
```

```

    Agachar();
}

```

• Regras / pegadinhas:

- **Caixa x função:** propriedade se atribui (`transform.localPosition = ...`), método se chama com parênteses (`Pular()`). Trocar um pelo outro dá CS1955 (chamou caixa) ou CS0428 / CS0201 (esqueceu os parênteses do método).
- Nome de alarme da Unity é senha; nome de método MEU é livre (`Pular` , `pular` , `DarUmPulo` — só ser consistente: PascalCase pra métodos, camelCase pra variáveis, é o padrão da Unity).
- Método `public` pode ser chamado pelo vizinho (`inimigo.TomarDano(1)`); `private` (ou sem nada) só de dentro do próprio script.
- `return;` sozinho num `void` = "sai agora". Ótimo guarda: `if (!estaNoChao) return;` no começo de `Pular()`.
- Variável declarada DENTRO de um método morre quando o método acaba. Pra durar entre quadros, ela sobe pra cima (campo da classe) — é por isso que `estaNoChao` fica no topo. E depois de subir, dentro dos métodos ela só é USADA pelo nome (`estaNoChao = true;`); escrever `bool estaNoChao = true;` de novo lá dentro cria uma segunda caixa com o mesmo nome — a sombra (1.11).

Exemplo: o Update picado em métodos (o mesmo comportamento, legível)

```

void Update()
{
    float lado = Input.GetAxisRaw("Horizontal");
    Andar(lado);
    Virar(lado);
    if (Input.GetButtonDown("Jump") && estaNoChao) Pular();
    if (Input.GetButtonUp("Jump")) CortarPulo();
}

void Andar(float lado)
{
    rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y); // irmã 1
}

void Pular()
{
    rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo); // irmã 2
    estaNoChao = false;
}

void CortarPulo()
{
    if (rb.linearVelocity.y > 0f)
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * 0.5f); // irmã 3
}

void Virar(float lado)
{
    if (lado != 0f) direcao = Mathf.Sign(lado);
    transform.localScale = new Vector3(direcao * escalaBase, transform.localScale.y, 1f);
}

```

1.7 Rigidbody2D e rb.linearVelocity — o motor físico

* Simples assim

O Rigidbody2D é o que dá "peso" ao boneco: com ele a gravidade puxa, a parede segura, o chão para. `rb.linearVelocity` é a velocidade de agora (x pro lado, y pra cima/baixo). Andar = escrever um x novo e deixar o y como está; pular = escrever um y novo e deixar o x como está. Por isso o `new Vector2(...)` sempre repete o eixo que não muda — é a "irmã" certa pra cada movimento.

```
rb.linearVelocity = new Vector2(5f, rb.linearVelocity.y); // anda pra direita; o y continua igual
```

- **O que é:** o componente que entrega o corpo ao motor de física (gravidade, inércia, colisão). `linearVelocity` é a caixa da velocidade: um `Vector2` (x = lado, y = vertical). Em versões antigas da Unity chama só `velocity`.
- **A receita das 3 irmãs (preservar o eixo alheio):** mexer num eixo SEM apagar o outro — o eixo que não é meu volta como está:
 - andar: `new Vector2(velocidade * lado, rb.linearVelocity.y)` (linha 19)
 - pular: `new Vector2(rb.linearVelocity.x, forcaPulo)` (linha 22)
 - cortar: `new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * 0.5f)` (linha 28)
- **Coleira sempre:** `linearVelocity` mora no `rb` → `rb.linearVelocity`, nunca solto (CS0103).

Mais fundo: as caixinhas do Rigidbody2D no Inspector

caixinha	o que faz	valor bom pra plataforma
Body Type	<code>Dynamic</code> = obedece física; <code>Kinematic</code> = só se mexe por código, ignora gravidade/força; <code>Static</code> = nunca se mexe	player/inimigo <code>Dynamic</code> ; plataforma móvel <code>Kinematic</code> ; chão <code>Static</code>
Mass	peso relativo em trombadas	1
Linear Damping (antes: <i>Drag</i>)	atrito com o ar — freia a velocidade sozinho	0 (eu cravo a velocidade)
Angular Damping	freio de rotação	0.05
Gravity Scale	multiplicador da gravidade só pra este corpo	2 a 4 (pulo "pesado" fica gostoso); 0 = flutua
Collision Detection	<code>Discrete</code> = checa por passo; <code>Continuous</code> = checa o caminho inteiro	<code>Continuous</code> se atravessa parede em alta velocidade
Sleeping Mode	corpo parado "dorme" pra economizar	<code>Start Awake</code>
Interpolate	suaviza o desenho entre passos da física	<code>Interpolate</code> no player (câmera para de tremer)
Constraints → Freeze Rotation Z	trava o giro	LIGAR no player — senão ele tomba e rola
Constraints → Freeze Position	trava um eixo	plataforma que só sobe/desce: trava X

No código: `rb.gravityScale = 3f; , rb.constraints = RigidbodyConstraints2D.FreezeRotation; , rb.bodyType = RigidbodyType2D.Kinematic; .`

Mais fundo: `linearVelocity` (cravar) × `AddForce` (empurrar)

jeito	o que faz	sensação	uso
<code>rb.linearVelocity = new Vector2(...)</code>	sobrescreve a velocidade agora	seco, previsível, responde na hora	andar, pular, dash — plataforma
<code>rb.AddForce(Vector2.up * 8f, ForceMode2D.Impulse)</code>	soma um tranco à velocidade atual	acumula, tem inércia	knockback, explosão, canhão
<code>rb.AddForce(Vector2.right * 20f)</code> (sem <code>Impulse</code>)	empurra aos pouquinhos, todo quadro	flutuante, "barco"	vento, correnteza

- Regra de bolso: o que o JOGADOR controla → cravar. O que o MUNDO faz com ele → empurrar.
- Pegadinha: se eu cravo `linearVelocity.x` todo quadro no `Update`, qualquer `AddForce` no x é apagado no quadro seguinte. Knockback precisa de uma janela em que o andar NÃO crava (um timer machucado — Parte 5.9).

Regras e pegadinhas extras

- **Física ignora o `transform`**: mudar `transform.position` teleporta o corpo (atravessa parede, perde a inércia). Corpo Dynamic se mexe por velocidade/força; posição na mão só em Kinematic (`rb.MovePosition`).
- Dois `Rigidbody2D` no mesmo objeto = erro. Um por objeto (o pai); os colliders dos filhos entram no mesmo corpo.
- Corpo Dynamic dorme se ficar parado; um `linearVelocity` acorda ele — não é problema meu.
- `rb.linearVelocity.y = 5f`; **não compila** (CS1612): `Vector2` é struct, tem que carimbar o vetor inteiro — é por isso que existem as 3 irmãs.
- `rb` sem arrastar no Inspector = `NullReferenceException` na primeira linha que usa. Alternativa: `rb = GetComponent<Rigidbody2D>();` no `Awake` (Parte 5.4) — aí nunca mais esqueço.
- Velocidade em `linearVelocity` é em **unidades por segundo**; 1 unidade = 1 metro = 100 pixels (com PPU 100 no sprite). `velocidade = 5f` = 5 m/s.

Formas de usar

```
// 1) Limitar a velocidade de queda (não virar bala ao cair de longe)
public float quedaMaxima = 20f;
void Update()
{
    if (rb.linearVelocity.y < -quedaMaxima)
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, -quedaMaxima);
}
```

```
// 2) Gravidade maior na queda = pulo com "peso" (sensação de jogo bom)
public float gravidadeSubida = 3f;
public float gravidadeQueda = 5f;
void Update()
{
    rb.gravityScale = rb.linearVelocity.y < 0f ? gravidadeQueda : gravidadeSubida;
}
```

```
// 3) Knockback: um empurrão pra longe de quem bateu
void Empurrar(Vector2 origem, float forca)
{
    Vector2 direcaoEmpurrao = ((Vector2)transform.position - origem).normalized;
```

```
rb.linearVelocity = Vector2.zero; // zera antes pra não somar com o andar
rb.AddForce(direcaoEmpurrao * forca, ForceMode2D.Impulse);
}
```

```
// 4) A receita "teclado no Update, física no FixedUpdate" (peça futura)
private float lado;
private bool querPular;

void Update()
{
    lado = Input.GetAxisRaw("Horizontal");
    if (Input.GetButtonDown("Jump") && estaNoChao) querPular = true; // só ANOTA
}

void FixedUpdate()
{
    rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y);
    if (querPular)
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo);
        querPular = false; // gasta a anotação
    }
}
```

1.8 transform — posição, escala, rotação, pai e filho

* Simples assim

Todo objeto tem um `transform`: onde está (`position`), que tamanho tem (`localScale`) e pra onde aponta (`rotation`). Escala x = -1 espelha o desenho — é o truque de virar o boneco. Mexer na posição direto é teleporte: o boneco aparece lá sem passar pelo caminho, e por isso atravessa parede. Pra andar de verdade, Rigidbody.

Todo `GameObject` tem um `Transform`, e dentro de qualquer script `transform` (minúsculo) é o crachá do objeto onde o script está pendurado. Ele guarda três caixas: **position**, **localScale**, **rotation**.

`transform.localScale` — tamanho e ESPELHO

- **O que é:** a escala do objeto (`Vector3`). Números maiores = maior; **x NEGATIVO espelha o sprite** — o flip de direção é `direcao * escalaBase` com `direcao` valendo +1 ou -1 (linha 52).
- **Ternário ? :** — if-else de uma linha: `float escalaY = pequeno ? escalaAgachado : escalaBase;` (lê-se: pequeno? então agachado, senão base).

`transform.localPosition` — propriedade é CAIXA, não função

(03/09/2026 — erro CS1955 no dash v1)

- **O que é:** a posição do objeto guardada numa caixa (propriedade), como `rb.linearVelocity` e `transform.localScale`. Caixa NÃO se chama com parênteses — se **atribui** com `=` e carimbo `new Vector3(x, y, z)`:
- errado: `transform.localPosition(x, y, 0);` → CS1955 "membro não invocável"
- certo: o mesmo formato da minha linha 51 (`localScale = new Vector3(...)`).
- **Pra que serve:** teleporte/reposicionamento direto — meu dash v1 soma `distanciaDash` no x.
- **Regras / pegadinhas:**
- Coleira sempre: dentro do carimbo é `transform.localPosition.x`, não `localPosition.x` solto (CS0103).

- **Teleporte ignora a física:** atravessa parede — mudar posição na mão não consulta colisor. Dash "de verdade" = velocidade + tempo (Parte 4.5).
- `if` sem chaves agarra SÓ a primeira linha seguinte; chaves vazias depois viram bloco órfão (prima do `if();`). Abraçar o corpo sempre.
- Dash com `+` fixo vai sempre pra direita — a variável que sabe pra onde olho (direcao) resolve com uma multiplicação.

Mais fundo: position ×.localPosition

caixa	mede a partir de	quando são iguais
<code>transform.position</code>	o (0,0) do MUNDO	sempre que o objeto não tem pai
<code>transform.localPosition</code>	a posição do PAI	—

Objeto sem pai: tanto faz. Objeto filho (ex.: `PontoDoGolpe` dentro do Boneco): `localPosition` é o deslocamento em relação ao Boneco, `position` é onde ele está no mundo. Pra `Instantiate` e distâncias, usar `position` (o mundo).

Mais fundo: rotação

```
transform.rotation = Quaternion.Euler(0f, 0f, 90f); // cravar 90° no z (2D gira só no z)
transform.Rotate(0f, 0f, 180f * Time.deltaTime); // girar 180° por segundo (moeda, hélice)
float angulo = transform.eulerAngles.z; // ler o ângulo atual em graus
```

`Quaternion.identity` = rotação zero, "reto do jeito que o molde é". Não tentar somar Quaternion na mão: usar `Euler` / `Rotate`.

Mais fundo: pai e filho (Hierarchy)

- Arrastar um objeto pra cima do outro na Hierarchy = vira **filho**. Filho anda, gira e escala junto com o pai.
- **Uso na Fase 2:** um objeto vazio `PontoDoGolpe` como filho do Boneco, um pouco à frente. Como o flip é `localScale.x = -1`, o ponto **espelha junto** — o golpe nasce na frente sem conta nenhuma: `Instantiate(golpePrefab, pontoDoGolpe.position, Quaternion.identity);`
- No código: `transform.parent`, `transform.SetParent(outro)`, `transform.GetChild(0)`, `transform.Find("PontoDoGolpe")`, `transform.childCount`.
- `Instantiate(prefab, posicao, rotacao, pai)` — 4º ingrediente opcional: já nasce como filho (golpe que acompanha o boneco enquanto existe).

Mais fundo: o pacote de contas do Vector2

escrita	resultado	uso
<code>Vector2.zero .one .up .down .left .right</code>	(0,0) (1,1) (0,1) (0,-1) (-1,0) (1,0)	atalhos prontos
<code>a + b</code> , <code>a - b</code>	soma/subtrai eixo a eixo	alvo - eu = o vetor que aponta pro alvo
<code>a * 2f</code>	dobra o tamanho	<code>Vector2.right * velocidade</code>
<code>a.magnitude</code>	comprimento do vetor	velocidade "real" ignorando direção

escrita	resultado	uso
<code>a.normalized</code>	mesma direção, comprimento 1	direção pura pra multiplicar por velocidade
<code>Vector2.Distance(a, b)</code>	distância entre dois pontos	"inimigo perto?"
<code>Vector2.Lerp(a, b, t)</code>	ponto entre a e b	suavizar
<code>Vector2.MoveTowards(a, b, passo)</code>	a andando pra b, no máximo passo	perseguir sem física
<code>(Vector2)transform.position</code>	converte Vector3 → Vector2	pra fazer contas com <code>position</code>

```
// Inimigo que anda na direção do player (sem física)
public Transform alvo;
public float velocidade = 2f;
void Update()
{
    transform.position = Vector2.MoveTowards(transform.position, alvo.position, velocidade * Time.deltaTime);
}
```

Regras e pegadinhas extras

- `transform.localScale` com x negativo espelha o **objeto inteiro**, filhos incluídos. É a vantagem sobre `SpriteRenderer.flipX`, que espelha só o desenho (Parte 4.7).
- Mudar `localScale` muda o **collider** junto (agachar encolhe a hitbox — bom) e espreme o sprite (feio). A versão profissional do agachar mexe no `BoxCollider2D` (Parte 4.6).
- Em 2D, o z da posição importa pra CÂMERA: a câmera fica em z = -10; um sprite em z = -11 fica atrás dela e some. Objetos do jogo ficam em z = 0.
- `transform.Translate(...)` também é teleporte: não consulta colisor. Bom pra coisas que não colidem (nuvem, moeda girando), ruim pro player.

1.9 Prefab — o molde azul (a peça 2, feita no editor)

* Simples assim

Prefab é o molde de biscoito: monto o objeto uma vez (sprite, collider, script), salvo na pasta (fica azul) e a Unity carimba cópias iguais quando eu pedir — golpes, moedas, inimigos. Mudou o molde, todas as cópias futuras mudam. Mudou uma cópia na cena, só ela muda (até eu clicar em *Apply*).

- **O que é:** um `GameObject` salvo como MOLDE na pasta Assets. Receita: montar o objeto na cena (componentes, script) → **arrastar da Hierarchy pra pasta Assets** → ícone fica AZUL = prefab → apagar o da cena.
- **Pra que serve:** o `Instantiate` carimba cópias dele; cada cópia nasce na cena com nome `(Clone)`.
- **Ligação com o código:** arrastar o prefab na caixinha `public GameObject` do Inspector — mesmo gesto do `rb`.

Mais fundo: editar o molde

- **Prefab Mode:** duplo-clique no prefab (ou a seta > ao lado do nome na Hierarchy) abre o molde sozinho, com fundo azul. Tudo que eu mudo ali vale pra TODAS as cópias futuras e pras que já estão na cena. Sair: seta < no topo da Hierarchy.

- **Override (o negrito azul):** mexer numa CÓPIA na cena deixa a caixinha em negrito com uma barra azul — mudou só nessa cópia. No topo do Inspector da cópia: *Overrides ▼ → Apply All* (manda pro molde) ou *Revert All* (volta pro molde).
- **Pegadinha:** ajustar a cópia da cena e achar que ajustou o prefab. O `Instantiate` carimba o MOLDE, não a cópia. Sempre aplicar (Apply) ou editar no Prefab Mode.
- **Caixinha de objeto dentro de prefab só aceita ASSET, não objeto da cena.** O molde vive na pasta; ele não sabe quem está na cena. Por isso a espada não pode ter uma caixinha "player" preenchida no prefab — ou quem carimba passa a referência (Parte 5.6), ou ela acha o player por tag (`GameObject.FindWithTag("Player")`).
- **Variante** (menu direito no prefab → *Create → Prefab Variant*): um molde "filho" que herda tudo do original e muda algumas coisas (inimigo vermelho = variante do inimigo).

Formas de usar

```
// O carimbo devolve a cópia — dá pra guardar e mexer nela na hora
GameObject copia = Instantiate(golpePrefab, transform.position, Quaternion.identity);
copia.name = "GolpeDoBoneco"; // tira o (Clone) do nome, só estética
Destroy(copia, 0.2f); // já agenda a borracha (Parte 5.5)
```

```
// Passar informação pra cópia recém-nascida (o jeito da peça 3.5 sem caixinha de vizinho)
GameObject copia = Instantiate(golpePrefab, transform.position, Quaternion.identity);
copia.GetComponent<Espada>().direcao = direcao; // a espada nasce sabendo pra onde ir
```

1.10 Debug.Log("...") — a prova no Console

* Simples assim

`Debug.Log("cheguei aqui")` é o bilhete que o código deixa no Console pra provar que passou por aquela linha. Não muda nada no jogo — é só pra eu ver. Quando algo "não funciona", o primeiro passo é botar um Log dentro do `if` ou do alarme suspeito: se o bilhete aparece, o código roda e o problema é outro; se não aparece, o código nunca chega ali.

```
if (estaNoChao)
{
    Debug.Log("estou no chão"); // apareceu? o if entra. Não apareceu? o problema é ANTES daqui
}
```

- Imprime no Console da Unity (aba Console, em casa). É o `print()` de lá: prova que um caminho do código rodou. O do stub não mostra nada — no trabalho ele só compila.

Mais fundo: a família

chamada	cor no Console	uso
<code>Debug.Log("x")</code>	branca	prova de execução, valores
<code>Debug.LogWarning("x")</code>	amarela	"isso não devia acontecer, mas segue"
<code>Debug.LogError("x")</code>	vermelha	erro meu, sem parar o jogo
<code>Debug.Log("x", gameObject)</code>	branca	clicar na linha do Console destaca o objeto na Hierarchy

chamada	cor no Console	uso
<code>Debug.DrawRay(origem, direcao, Color.red)</code>	—	desenha um raio na aba Scene (Gizmos ligado) por 1 quadro
<code>Debug.DrawLine(a, b, Color.green, 2f)</code>	—	linha entre dois pontos, por 2 segundos
<code>Debug.Break()</code>	—	pausa o jogo nessa linha (como apertar Pause)
<code>print("x")</code>	branca	sinônimo de <code>Debug.Log</code> dentro de <code>MonoBehaviour</code>

Mais fundo: escrever valores

```

Debug.Log("vel y: " + rb.linearVelocity.y);           // concatenar com +
Debug.Log($"vel y: {rb.linearVelocity.y:F2}");        // $ + chaves; :F2 = 2 casas decimais
Debug.Log(gameObject.name + " encostou em " + colisao.gameObject.name);
Debug.Log(estaNoChao);                               // "True"/"False"

```

Regras e pegadinhas

- `Debug.Log` dentro do `Update` = 60 linhas por segundo. Ligar **Collapse** no Console (junta as repetidas com um contador) ou logar só dentro de um `if` /alarme.
- **Console** → duplo-clique numa linha abre o VS Code na linha exata. Vale pra erro e pra Log.
- Botões da barra do Console: **Clear** (limpa), **Collapse** (junta iguais), **Clear on Play** (limpa ao dar Play — ligar), **Error Pause** (pausa o jogo no primeiro erro — ótimo pra pegar `NullReference` no ato), e os três filtros do canto (mensagem / aviso / erro — se sumiu tudo, um filtro está desligado).
- `Debug.DrawRay` só aparece na aba **Scene**, com o botão *Gizmos* ligado; na aba Game não.
- Log demais pesa. Antes de exportar o jogo, apagar ou comentar (`// Debug.Log(...)`).
- Pra ver a Hierarchy/Inspector mudando em tempo real, deixar a aba Game de um lado e a Scene do outro (aba Game → *Maximize On Play* desligado).

1.11 Limpeza e estruturação do código — uma caixa, um dono; declarar uma vez só [peça em uso, aula de 04/09]

(04/09/2026 — a faxina do `Movimento.cs` depois que o ground check virou círculo)

* Simples assim

É como ter dois relógios na parede marcando horas diferentes: nunca sei em qual olhar. Se o `Update` já calcula `estaNoChao` todo quadro, o alarme de colisão não pode escrever nela também — fica UM relógio só. E declarar de novo com `bool` dentro do método é como comprar uma segunda caixa com a mesma etiqueta: guardo a coisa na caixa nova, jogo a caixa fora no fim do método, e a caixa de verdade (a do topo) continua como estava.

- **O que aconteceu:** ao trocar o ground check pra `Physics2D.OverlapCircle` (3.5), o `Movimento.cs` ficou com **duas fontes de verdade** pra `estaNoChao`: o `Update` calculava ela todo quadro E os alarmes `OnCollisionEnter2D` / `OnCollisionExit2D` continuavam escrevendo `true` / `false` nela. Resultado: briga. Um liga, o outro desliga, o valor pisca e o pulo "às vezes funciona".
- **A regra: uma variável, um dono.** Se o `Update` calcula `estaNoChao` a cada quadro, os alarmes **NÃO** mexem nela. O alarme de colisão só fica no script se tiver outro trabalho (som de pouso, dano de espinho, quebrar caixa) — senão, apaga.

- **O erro de sintaxe que apareceu junto:** na hora de "consertar", a variável foi declarada de novo dentro do método. Campo declarado no topo da classe **não se redeclara** dentro de método — nem com tipo (`bool`), nem com modificador de acesso (`public`):
- `public bool estaNoChao = true;` dentro de um método → **CS0106** ("o modificador 'public' não é válido para este item"). Erro barulhento: `public/private` só existem no topo da classe.
- `bool estaNoChao = true;` dentro de um método → **compila** — e é pior. Cria uma gaveta NOVA, local, com o mesmo nome (a **sombra**). O método guarda o valor nessa gaveta nova, ela morre no fim do método, e a caixa do topo (a que o `Update` lê) fica intocada. Bug silencioso: "mudo e nada muda".
- `estaNoChao = true;` dentro do método → **certo**: só o nome, sem tipo. É a caixa do topo que muda.

Mais fundo: onde cada linha pode aparecer

linha	no topo da classe	dentro de um método
<code>public bool estaNoChao;</code>	✓ declara a caixinha (aparece no Inspector)	✗ CS0106
<code>private bool estaNoChao;</code>	✓ declara a gaveta	✗ CS0106
<code>bool estaNoChao;</code>	✓ gaveta (sem nada na frente = <code>private</code>)	⚠ compila, mas cria uma variável local NOVA com o mesmo nome — sombra
<code>estaNoChao = true;</code>	✗ CS1519 (instrução solta fora de método)	✓ muda a caixa do topo
<code>bool apertou = Input.GetButtonDown("Jump");</code>	✗	✓ variável local de verdade: nome NOVO, vive só até o fim do método

Regra de bolso: **tipo + nome = declarar (uma vez, no topo)**. **Só o nome = usar (quantas vezes quiser, dentro dos métodos)**. Variável local (declarada dentro do método) é ótima — desde que tenha um nome que NÃO exista no topo.

O antes e o depois (a faxina de 04/09)

```
// ANTES – três lugares mexendo na mesma caixa (e uma sombra no meio)
private bool estaNoChao;

void Update()
{
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao); // dono 1
    if (Input.GetButtonDown("Jump") && estaNoChao) Pular();
}

void OnCollisionEnter2D(Collision2D colisao)
{
    bool estaNoChao = true; // dono 2 – e SOMBRA: essa é OUTRA caixa; a do topo nem muda
}

void OnCollisionExit2D(Collision2D colisao)
{
    estaNoChao = false; // dono 3 – desliga o que o Update acabou de ligar → pisca
}
```

```
// DEPOIS – um dono só; os alarmes que não tinham outro trabalho foram embora
private bool estaNoChao;

void Update()
{
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao); // só aqui
```

```
if (Input.GetButtonDown("Jump") && estaNoChao) Pular();  
}
```

Regras e pegadinhas

- Antes de apagar um alarme, perguntar: "ele faz mais alguma coisa além de mexer em `estaNoChao`?". Se sim (som de pouso, dano), fica — só sem a linha do `estaNoChao`.
- A sombra **não dá erro vermelho**: o compilador aceita porque o nome novo vive só dentro das chaves do método. Regra visual: se dentro de um método aparece `bool`, `float`, `int` na frente de um nome que já existe no topo, é sombra.
- Sintoma da sombra: `Debug.Log(estaNoChao)` no `Update` nunca muda, mas dentro do alarme mostra `True`. São duas caixas.
- Estruturar por método ajuda a enxergar isso: o `Update` só orchestra (chama `ChecarChao()`, `Andar()`, `Pular()`, `Animar()` — 1.6); cada método mexe nas SUAS caixas. Se dois métodos escrevem na mesma caixa, um deles está errado — ou ela precisa de um dono só.
- O mesmo vale pra qualquer caixa que o `Update` recalcula: `direcao`, `dando`, `agachado`. Quem calcula é dono; o resto só lê.

PARTE 2 — INPUT: CAMPAINHAS, RÉGUAS E TECLAS CRUAS

2.1 Input.GetButtonDown("Jump") — a campainha de APELIDO

(03/09/2026 — usada em `aula_8/Assets/Scripts/Movimento.cs`, linha 19)

* Simples assim

"Jump" não é a tecla: é um **apelido** que a Unity liga a uma tecla (espaço, de fábrica).

`Input.GetButtonDown("Jump")` pergunta "apertaram o apelido Jump agora?" — só responde sim no exato quadro em que o dedo desceu. A vantagem: se amanhã o pulo for no W, mudo na lista da Unity, sem mexer no código.

- **O que é:** a Unity tem uma lista de botões batizados com apelido em Edit → Project Settings → **Input Manager**: "Jump", "Horizontal", "Fire1"... Cada apelido aponta pra teclas reais ("Jump" → espaço, "Horizontal" → A/D e setas). `GetButtonDown` pergunta: "o apelido tal foi apertado NESTE quadro?"
- **Pra que serve:** desacoplar a ação da tecla. Quiser o pulo no W, ou num controle de Xbox? Muda **na lista** do Input Manager, sem tocar no código.
- **Regras / pegadinhas:**
 - Entre as aspas só entra **apelido que existe na lista**. "s" não é apelido de nada → a Unity **não dá erro, só fica muda pra sempre**. Erro silencioso.
 - Pra perguntar por uma **tecla crua**, sem apelido, a porta é outra: `Input.GetKeyDown(KeyCode.S)`.
 - **A família** (vale pros dois mundos, Button e Key):

forma	pergunta	dispara
<code>...Down</code>	apertou AGORA, neste quadro?	1 vez por aperto
sem sufixo (<code>GetButton</code> / <code>GetKey</code>)	está SEGURANDO?	todo quadro enquanto segura
<code>...Up</code>	soltou AGORA?	1 vez, no soltar

- **No meu código:** linha 19 usa apelido (`GetButtonDown("Jump")` no pulo); linhas 29 e 33 usam tecla crua (`GetKeyDown` / `GetKeyUp` com `KeyCode.S` no agachar).

Formas de usar

```
if (Input.GetButtonDown("Jump")) Pular();           // 1 vez por aperto
if (Input.GetButton("Fire1"))   AtirarContinuo();  // metralhadora: todo quadro segurando
if (Input.GetButtonUp("Jump"))  CortarPulo();       // no soltar
```

```
// Um aperto que vale pra duas coisas dependendo do estado
if (Input.GetButtonDown("Fire1"))
{
    if (estaNoChao) AtaqueNoChao();
    else           AtaqueAereo();
}
```

2.2 A lista de apelidos de fábrica (Input Manager)

(03/09/2026 — a lista que faltou na peça do Jump)

* Simples assim

É a tabela de apelidos que já vem pronta: Jump, Fire1, Horizontal... Cada nome é uma senha: tem que ser escrito igualzinho, com maiúscula. "jump" minúsculo compila e só explode com o jogo rodando. Botão "não funciona"? Conferir a grafia nesta tabela antes de qualquer outra coisa.

CAMPAINHAS — respondem sim/não → família `GetButton/Down/Up` :

apelido	tecla de fábrica
"Jump"	Espaço
"Fire1"	Ctrl esq. (+ <i>clique esquerdo</i>)
"Fire2"	Alt esq. (+ <i>clique direito</i>)
"Fire3"	Shift esq. (+ <i>clique do meio</i>)
"Submit"	Enter
"Cancel"	Esc

RÉGUAS — respondem um NÚMERO → família `GetAxis/GetAxisRaw` :

apelido	o que mede
"Horizontal"	A/D e ↔ (-1, 0, +1 no Raw)
"Vertical"	W/S e ↑ ↓ (idem)
"Mouse X" / "Mouse Y"	movimento do mouse
"Mouse ScrollWheel"	a rodinha

- **Apelido em string é senha exata que o compilador NÃO CONFERE:** "fire1" compila feliz e explode em casa rodando ("Input Button fire1 is not setup"). Os 3 tipos de erro: compilação (grita no build), silencioso (nome de evento errado — nunca chama), e de STRING (só explode rodando). Conferir maiúsculas na tabela acima.
- **Receita de COMBO (ex.: deslizada = S + Dash):** dois `Down` juntos exigem os dois apertos no MESMO quadro (1/60s) — dedo humano não acerta. O modificador fica SEGURADO (`GetKey`, sem sufixo) e só o gatilho usa `Down`: `if (Input.GetButtonDown("Dash") && Input.GetKey(KeyCode.S))`.
- **Onde ver em casa:** Edit → Project Settings → **Input Manager** → abrir "Axes". Cada apelido com suas teclas, duplicado pra controle de videogame.
- **A lista é minha:** dá pra criar apelido novo lá (um "Dash" no futuro — receita em 2.6).
- **Campainha ≠ régua:** régua se lê com `GetAxisRaw` (minha linha 17), não com `GetButtonDown`.

2.3.GetAxis × GetAxisRaw — a régua com e sem amortecimento [peça de apoio]

* Simples assim

Os dois leem a mesma coisa (A/D ou setas) e devolvem um número: -1 esquerda, 0 parado, +1 direita. `GetAxis` suaviza (o número sobe devagar, como um carro acelerando); `GetAxisRaw` vai direto ao -1/0/+1 (como um botão). Boneco de plataforma que responde na hora usa `Raw`. Se ele "escorrega" depois que solto a tecla, é o `GetAxis` suavizando.

- **O que é:** as duas leem a mesma régua, mas `GetAxis` **suaviza**: ao apertar D, o valor sobe 0 → 0.3 → 0.7 → 1 ao longo de alguns quadros, e ao soltar desce devagar. `GetAxisRaw` devolve **-1, 0 ou +1 na hora** (pra teclado).
- **Pra que serve:** `GetAxisRaw` = controle seco de plataforma (o que uso). `GetAxis` = carro, nave, coisas com aceleração; e pra analógico de controle ele devolve o valor real (0.37...).
- **Regras / pegadinhas:**
 - "Meu boneco desliza depois que solto a tecla" = `GetAxis`. Trocar pra `Raw`.
 - Os números do amortecimento moram no Input Manager: **Gravity** (quão rápido volta a 0), **Sensitivity** (quão rápido vai a 1), **Dead** (zona morta do analógico), **Snap** (inverter direção zera na hora).
 - Régua devolve float: `lado == 1` funciona no teclado, mas com analógico pode ser 0.8. Perguntar `lado != 0f` ou `Mathf.Abs(lado) > 0.01f`.
 - Régua lida com `GetButtonDown("Horizontal")` funciona (os eixos também são botões), mas devolve só sim/não — perde o lado.

Formas de usar

```
float lado = Input.GetAxisRaw("Horizontal"); // -1, 0, +1
float vert = Input.GetAxisRaw("Vertical"); // -1, 0, +1

bool andando = lado != 0f; // pra animação
if (lado != 0f) direcao = Mathf.Sign(lado); // guarda a última direção olhada

// Nave top-down: os dois eixos de uma vez, normalizado pra diagonal não ser mais rápida
Vector2 entrada = new Vector2(lado, vert).normalized;
rb.linearVelocity = entrada * velocidade;
```

2.4.Input.GetKey e a tabela de KeyCode [peça de apoio]

* Simples assim

`GetKey` pergunta pela tecla física, sem apelido: `Input.GetKeyDown(KeyCode.S) = "apertou o S agora?"`. `KeyCode` é a lista de todas as teclas; como é uma lista fixa, se eu escrever errado (`KeyCode.Espaco`) o compilador avisa na hora — melhor que a string do apelido, que só reclama rodando.

- **O que é:** a porta da tecla crua. Mesma família Down / segurando / Up. O parâmetro é um `KeyCode` — uma lista fixa que o compilador CONFERE (`KeyCode.Espaco` não existe → `CS0117`, erro barulhento, melhor que string).
- **Os KeyCodes que mais aparecem:**

tecla	KeyCode	tecla	KeyCode
letras	KeyCode.A ... KeyCode.Z	espaço	KeyCode.Space
números de cima	KeyCode.Alpha0 ... Alpha9	numérico	KeyCode.Keypad0 ... Keypad9
setas	UpArrow DownArrow LeftArrow RightArrow	Enter	KeyCode.Return
Shift	LeftShift / RightShift	Ctrl	LeftControl / RightControl
Alt	LeftAlt / RightAlt	Esc	KeyCode.Escape
Tab	KeyCode.Tab	Backspace	KeyCode.Backspace
F1...F12	KeyCode.F1 ... F12	mouse	Mouse0 (esq) Mouse1 (dir) Mouse2 (meio)
controle	JoystickButton0 ... JoystickButton19	qualquer tecla	Input.anyKeyDown (bool)

• Regras / pegadinhas:

- `KeyCode.A` é a **tecla física A**, não a letra; com Caps Lock ou Shift continua sendo A.
- `Alpha1` é o 1 da fileira de cima; `Keypad1` é o do teclado numérico. Não são a mesma tecla.
- Apelido (`GetButton`) é melhor pra ações do jogo (pular, atacar); tecla crua é boa pra atalho fixo (Esc = menu, R = reiniciar, F1 = debug).

```
if (Input.GetKeyDown(KeyCode.Escape)) AbrirMenu();
if (Input.GetKeyDown(KeyCode.R))      ReiniciarFase();
if (Input.GetKey(KeyCode.LeftShift))  correndo = true;    // segurando = correr
```

2.5 Mouse — clique e posição [peça futura]

* Simples assim

O mouse fala com o jogo de dois jeitos: os botões (`GetMouseButtonDown(0)` é o clique esquerdo) e a posição. A posição vem em pixels da tela, e o mundo do jogo é medido em unidades — então antes de usar eu converto com `Camera.main.ScreenToWorldPoint`. É como passar centímetros pra metros antes de fazer a conta.

- **Clique:** `Input.GetMouseButtonDown(0)` (esquerdo), (1) direito, (2) meio. Mesma família `GetMouseButton` / `GetMouseButtonUp`.
- **Posição:** `Input.mousePosition` é em **pixels da tela**, origem no canto inferior esquerdo. Pra virar posição do mundo (onde o boneco vive): `Camera.main.ScreenToWorldPoint(...)` e zerar o z.
- **Rodinha:** `Input.GetAxis("Mouse ScrollWheel")` (positivo = pra frente).

```
// Atirar na direção do mouse
if (Input.GetMouseButtonDown(0))
{
    Vector3 mundo = Camera.main.ScreenToWorldPoint(Input.mousePosition);
    mundo.z = 0f;                                     // senão fica no z da câmera
    Vector2 direcaoTiro = ((Vector2)mundo - (Vector2)transform.position).normalized;
    GameObject bala = Instantiate(balaPrefab, transform.position, Quaternion.identity);
}
```

```
bala.GetComponent<Rigidbody2D>().linearVelocity = direcaoTiro * 15f;
}
```

- **Pegadinha:** `Camera.main` procura a câmera com a tag **MainCamera** — a Main Camera da cena já vem com ela; se eu criar outra câmera, colar a tag.

2.6 Criar um apelido novo no Input Manager — passo a passo do "Dash" [peça futura]

* Simples assim

Criar um apelido é cadastrar um botão novo na tabela da Unity: nome, tecla, pronto. Depois disso `Input.GetButtonDown("Dash")` funciona igual ao `"Jump"`. O passo que todo mundo esquece: Ctrl+S salva só a cena; a tabela de apelidos só vai pro disco com **File → Save Project**.

1. **Edit → Project Settings → Input Manager**, abrir **Axes**.
2. No campo **Size** (topo da lista), somar 1 ao número. A Unity **clona o último apelido** da lista no fim — é esse clone que vou editar (rolar até o fim).
3. Abrir o novo item e preencher:
 - **Name:** `Dash` (é a senha exata que vai no `GetButtonDown("Dash")` — maiúscula igual).
 - **Positive Button:** `left shift` — os nomes de tecla aqui são **minúsculos com espaço**: `space`, `left ctrl`, `left shift`, e, `mouse 0`, `joystick button 0`.
 - **Alt Positive Button:** `joystick button 1` (o mesmo apelido no controle).
 - **Type:** `Key or Mouse Button`.
 - **Gravity** `1000`, **Dead** `0.001`, **Sensitivity** `1000` (valores de campanha; com esses, a régua vira 0/1 na hora).
 - **Negative Button** e **Snap/Invert** ficam vazios/desmarcados pra campanha.
4. **File → Save Project**. Ctrl+S salva só a CENA; as configurações do projeto (Input Manager, Tags, Layers) só vão pro disco — e pro git — com *Save Project*.
5. No código: `if (Input.GetButtonDown("Dash") && podeDar) Dash();`
 - **Pegadinha:** dois apelidos com o mesmo nome são permitidos (teclado + controle é exatamente isso); a Unity responde sim se QUALQUER um disparar.
 - **Pegadinha:** o Input Manager é global do projeto. No trabalho, com o stub, ele não existe — só em casa dá pra testar.

2.7 Aviso de fronteira: o "novo Input System" [só pra reconhecer]

* Simples assim

Existem dois sistemas de teclado na Unity: o clássico (este manual inteiro) e o "novo Input System". Eles não se misturam. Se um tutorial mostra `using UnityEngine.InputSystem;`, é do outro mundo — não copiar. Se aparecer um erro dizendo que o projeto está no outro sistema, o conserto está aqui embaixo.

- Existe um pacote chamado **Input System** (com Action Maps, `PlayerInput`, `InputAction`). Tudo deste manual é o **Input Manager clássico** (`Input.GetButtonDown`).

- **O erro que denuncia:** `InvalidOperationException: You are trying to read Input using the UnityEngine.Input class, but you have switched active Input handling to Input System package.`

Conserto: Edit → Project Settings → **Player** → *Other Settings* → **Active Input Handling** → Input Manager (Old) ou Both. A Unity reinicia o editor.

- Migrar pro novo é peça pra muito depois. Tutoriais na internet misturam os dois — se aparecer `using UnityEngine.InputSystem;` no exemplo, é o outro mundo.

PARTE 3 — FÍSICA 2D: COLLIDERS, TROMBADAS E TOQUES FANTASMAS

3.1 Colliders — a casca que a física enxerga [peça de apoio]

* Simples assim

O desenho (sprite) é só tinta; quem "existe" pra física é o collider, uma caixa invisível em volta do boneco. Sem ela, o boneco atravessa tudo. Cada objeto sólido precisa da sua: boneco, chão, parede. Dica de ouro: a casca aparece em **verde** na aba Scene quando o objeto está selecionado — se o verde não bate com o desenho, é ali que se ajusta.

- **O que é:** o **sprite é só desenho**; quem trombada é o **collider**, uma casca invisível. Sem collider, o boneco cai pelo chão. Sem collider no chão, idem.
- **Os tipos:**

componente	forma	usar em
BoxCollider2D	retângulo	player, caixas, plataformas, chão
CircleCollider2D	círculo	moeda, bola, ponto do pé (ground check)
CapsuleCollider2D	cápsula (retângulo com pontas redondas)	player que "engancha" em quinas — a cápsula desliza
PolygonCollider2D	segue o contorno do sprite	terreno irregular, objeto com forma esquisita (pesado)
EdgeCollider2D	uma linha	chão fininho, rampa
CompositeCollider2D	funde vários em um	tilemap de chão (junto com o TilemapCollider2D)

- **As caixinhas que importam:** **Offset** (deslocar a casca sem mexer o objeto), **Size/Radius** (tamanho), botão **Edit Collider** (arrastar as bordas na Scene), **Is Trigger** (casca sólida × fantasma — Parte 3.4), **Material** (Physics Material 2D).
- **Physics Material 2D** (Assets → Create → 2D → Physics Material 2D): **Friction** (atrito: 0 = gelo) e **Bounciness** (0 = não quica; 1 = bola). Arrasta na caixinha *Material* do collider.
- **Regras / pegadinhas:**
- **Boneco gruda na parede no meio do pulo** = atrito entre os colliders. Material com Friction 0 no collider do player resolve (receita abaixo — aula de 04/09).
- Collider maior que o sprite = "bate no ar". Menor = atravessa visualmente. *Edit Collider* e conferir na Scene com o objeto selecionado (a casca aparece em verde).
- Objeto com Rigidbody2D + vários colliders nos filhos = um corpo só, feito de várias cascas (corpo + cabeça, por exemplo).
- Dois colliders no MESMO objeto: pode (um sólido pro chão, um trigger pra detectar), mas cada alarme vem sem dizer qual casca disparou — melhor separar em filhos.
- Collider sem Rigidbody2D em nenhum dos dois = parede contra parede: nada acontece, nenhum alarme.

Fricção zero — o boneco para de grudar na parede [aula de 04/09]

1. Project → botão direito na pasta Assets → *Create* → *2D* → *Physics Material 2D* → nome `SemAtrito`.
2. Selecionar o `SemAtrito` → Inspector: **Friction** = 0, **Bounciness** = 0.
3. Selecionar o **Boneco** → componente `BoxCollider2D` (ou `Capsule`) → caixinha **Material** → arrastar o `SemAtrito`.
4. Play: pular contra uma parede segurando a direção pra ela. O boneco tem que **escorregar** até o chão sem parar no meio. Se ainda gruda, o material foi parar no chão em vez do collider do boneco — ou o ground check ainda é por trombada (3.5).

- **Por que no boneco e não nas paredes:** o atrito é calculado entre os DOIS colliders; com 0 num deles, todo contato vira "gelo" — resolve pra todas as paredes do jogo de uma vez. O chão continua segurando o boneco pela gravidade, não pelo atrito; a velocidade de andar é cravada pelo código (1.7), então ele não escorrega parado.
- **Pegadinha:** com atrito 0, plataforma móvel não "carrega" o boneco pelo atrito — resolve-se por pai/filho ou somando a velocidade da plataforma (peça futura).

3.2 OnCollisionEnter2D(Collision2D colisao) — o alarme de trombada

(03/09/2026 — usada em `aula_8/Assets/Scripts/Movimento.cs`, linhas 40-46)

* Simples assim

É a campanha física: toca sozinha, uma vez, quando a casca deste objeto encosta em outra. Eu não chamo; a Unity chama e entrega um "boletim" (`colisao`) dizendo quem encostou. Detalhe importante (o que me pegou na parede): a campanha toca pra **qualquer lado** da casca — em cima, embaixo ou de lado. Por isso ela sozinha não serve pra saber se estou no chão (3.5).

- **O que é:** método de **evento**, família do `Update`: eu nunca chamo — **a Unity chama por mim**, no quadro em que o collider deste objeto ENCOSTA em outro collider. A família completa:

método	a Unity chama quando	frequência
<code>OnCollisionEnter2D</code>	encostou AGORA	1 vez, no toque
<code>OnCollisionStay2D</code>	continua encostado	todo quadro do contato
<code>OnCollisionExit2D</code>	desencostou AGORA	1 vez, no soltar

(mesmo trio Down / segurando / Up das teclas, só que pra toques físicos)

- **O `Collision2D colisao` :** o **boletim de ocorrência** da trombada — a Unity preenche e entrega: quem encostou (`colisao.gameObject`), onde, com que força. Regra de sempre: assinatura DECLARA tipo + apelido, corpo USA o apelido (o apelido é meu: usei `collision`, valia qualquer nome).
- **Pra que serve:** reagir ao mundo físico — pisar no chão (meu ground check), tomar dano em espinho, quebrar caixa.
- **Regras / pegadinhas:**
 - O nome é **senha exata**, com o **2D no fim** (a versão sem 2D é do mundo 3D). Errou uma letra? **Sem erro nenhum — a Unity só nunca chama**. Erro silencioso, o mesmo veneno do `"s"` no `GetButtonDown`.
 - Só dispara se os DOIS objetos têm collider E pelo menos um tem **Rigidbody2D** (normalmente o que se mexe).

- `CompareTag("Ground")` : Tag é a etiqueta colada no objeto no topo do Inspector. Precisa **existir na lista E estar colada no chão** — chão sem etiqueta = o alarme toca mas o `if` ignora.
- **No meu código (histórico):** até 03/09 o ground check morava aqui — o Enter ligava `estaNoChao` e o pulo desligava (linha 22). Tinha dois furos: cair da beirada SEM pular deixava o chão ligado (pulo no ar durante a queda), e **encostar na PAREDE no meio do pulo também ligava** — o alarme toca pra qualquer lado do collider, inclusive as laterais, então o boneco pulava de novo grudado na parede. **Em 04/09 o ground check saiu daqui** e virou `Physics2D.OverlapCircle` nos pés (3.5); este alarme fica reservado pra dano, som de pouso, caixa que quebra.

Mais fundo: o que vem dentro do boletim

membro	o que é	uso
<code>colisao.gameObject</code>	o OUTRO objeto (o que encostou em mim)	<code>CompareTag</code> , <code>GetComponent</code> , <code>.name</code>
<code>colisao.collider</code>	o collider do outro	saber qual casca, se ele tem várias
<code>colisao.otherCollider</code>	o MEU collider que encostou	idem, do meu lado
<code>colisao.contactCount</code>	quantos pontos de contato	0 = raro, mas guardar
<code>colisao.GetContact(0).point</code>	onde encostou (mundo)	partícula de poeira no pé
<code>colisao.GetContact(0).normal</code>	a direção "pra fora" da superfície	chão ou parede? <code>normal.y > 0.5</code> = chão
<code>colisao.relativeVelocity</code>	com que velocidade bateram	dano proporcional à queda

Formas de usar

```
// 1) Tapar o furo da beirada: o Exit desliga o chão
void OnCollisionExit2D(Collision2D colisao)
{
    if (colisao.gameObject.CompareTag("Ground")) estaNoChao = false;
}
```

```
// 2) Encostar na PAREDE de lado não pode contar como chão: olhar a normal
void OnCollisionEnter2D(Collision2D colisao)
{
    if (colisao.gameObject.CompareTag("Ground") && colisao.GetContact(0).normal.y > 0.5f)
        estaNoChao = true;
}
```

```
// 3) Espinho: dano no toque
void OnCollisionEnter2D(Collision2D colisao)
{
    if (colisao.gameObject.CompareTag("Espinho")) TomarDano(1);
}
```

```
// 4) Caixa que quebra quando algo pesado cai em cima
void OnCollisionEnter2D(Collision2D colisao)
{
    if (colisao.relativeVelocity.magnitude > 8f) Destroy(gameObject);
}
```

Mais fundo: quem recebe o alarme (a tabela dos corpos)

A encosta em B	A é...	B é...	alarme?
player cai no chão	Dynamic	Static (sem rb)	sim , nos dois scripts
player bate no inimigo	Dynamic	Dynamic	sim , nos dois
plataforma Kinematic passa por parede	Kinematic	Static	não — dois "não-dinâmicos" não trombam
golpe trigger toca inimigo	Kinematic + trigger	Dynamic	sim (Trigger) — 3.4
moeda trigger, ninguém tem rb	sem rb + trigger	sem rb	não — precisa de um rb

O alarme chega **nos dois objetos**: o script do player recebe "encostei no chão" e o script do chão (se tiver) recebe "o player encostou em mim". Quem trata o dano decide o projeto — regra da Fase 2: quem NASCE (a espada) fala; quem VIVE (o inimigo) tem o método `TomarDano`.

3.3 Tags e Layers — etiquetas e camadas [peça de apoio]

* Simples assim

Tag é uma etiqueta com nome colada no objeto (`Player`, `Ground`) — serve pro código perguntar "quem é você?". Layer é a "turma" do objeto — serve pra dizer quem colide com quem e pra filtrar o que o círculo do ground check pode enxergar. Os dois têm o mesmo passo esquecido: criar o nome na lista **não** cola no objeto; tem que voltar no objeto e escolher.

Tag — a etiqueta

- **O que é:** um texto colado no objeto (topo do Inspector, dropdown `Tag`). Um objeto tem UMA tag. Vem de fábrica: `Untagged`, `Respawn`, `Finish`, `EditorOnly`, `MainCamera`, `Player`, `GameController`.
- **Criar e colar (dois passos — o segundo é o que esquece):** 1. Selecionar o objeto → `Tag` → **Add Tag...** → na lista `Tags`, + → digitar `Ground` → Enter (Save, se aparecer). 2. **Voltar ao objeto** → `Tag` → escolher `Ground`. Criar a tag NÃO cola nela: ela nasce na lista, e o objeto continua `Untagged`.
- **No código:** `colisao.gameObject.CompareTag("Ground")`. Também existe `colisao.gameObject.tag == "Ground"` — funciona, mas `CompareTag` é mais rápido e, no editor, **grita** se a tag nem existir na lista (`Tag: ground is not defined`) — erro barulhento, melhor que o `==` mudo.
- **Achar por tag:** `GameObject.FindWithTag("Player")` devolve o primeiro objeto com a tag (ou `null`). Bom no `Start` da espada/inimigo pra achar o player sem caixinha.
- **Pegadinhas:** senha exata (`"ground" ≠ "Ground"`); prefab novo nasce `Untagged`; tag no pai não desce pros filhos (o pé do boneco não é `Player` a menos que eu cole).

Layer — a camada

- **O que é:** um número de 0 a 31 (dropdown `Layer`, ao lado da `Tag`). Serve pra dizer **quem colide com quem** e pra filtrar raios (`LayerMask`). Camadas de fábrica: `Default`, `TransparentFX`, `Ignore Raycast`, `Water`, `UI`. As minhas começam na **User Layer 6**.
- **Criar:** `Layer` → **Add Layer...** → escrever `Chao`, `Player`, `Inimigo`, `Golpe` nas linhas vazias → voltar ao objeto → escolher a camada. Ao mudar a layer de um pai, a Unity pergunta se muda os filhos (`Yes, change children`) — normalmente sim.

- **Layer Collision Matrix:** Edit → Project Settings → **Physics 2D** → aba *Layer Collision Matrix*. Desmarcar a caixinha `Golpe × Player` = a espada do boneco nunca acerta o próprio boneco; `Inimigo × Inimigo` desmarcado = inimigos atravessam uns aos outros sem empurrar.
- **No código:** `public LayerMask camadaChao;` vira uma caixinha com checkboxes no Inspector. Vai no `Physics2D.OverlapCircle(..., camadaChao)` / `Physics2D.Raycast(..., camadaChao)` (3.5).
- **Pegadinhas:** `LayerMask` vazia no Inspector = nenhum raio acerta nada = `estaNoChao` nunca liga; `gameObject.layer = 6` é o número, não o nome — pra usar nome: `LayerMask.NameToLayer("Chao")`.

3.4 OnTriggerEnter2D(Collider2D outro) — o toque fantasma [peça 4 do roadmap]

* Simples assim

Trigger é a casca fantasma: não empurra nem para ninguém, mas avisa quando alguém entra. Moeda, golpe da espada, checkpoint — tudo o que precisa "sentir" sem "bater". Ligo o *Is Trigger* no collider e a campainha muda de nome: `OnTriggerEnter2D`, com um boletim menor (`Collider2D` outro).

- **O que é:** o primo do `OnCollisionEnter2D` pra colliders com **Is Trigger** ligado: a casca fantasma **não empurra nem para** ninguém, mas **avisa** quem entrou nela. Família idêntica: `OnTriggerEnter2D` (entrou agora), `OnTriggerStay2D` (está dentro), `OnTriggerExit2D` (saiu agora).
- **O parâmetro é outro:** `Collider2D` outro, não `Collision2D`. Toque fantasma não tem força nem ponto de contato — o boletim é só "quem". `outro.gameObject`, `outro.CompareTag("...")`, `outro.GetComponent<Inimigo>()`.
- **Pra que serve:** tudo que **atravessa** mas precisa **detectar**: golpe da espada, moeda, área de dano, checkpoint, porta que abre quando chego, zona de morte no fundo da fase.
- **Regras / pegadinhas:**
 - Senha exata com `Trigger` e 2D. `OnTriggerEnter` (sem 2D) = mundo 3D = silencioso.
 - Igual ao Collision: pelo menos um dos dois precisa de **Rigidbody2D**. Golpe/moeda sem rb tocando o player (que tem rb) funciona. Moeda parada sem rb tocando chão sem rb: nada.
 - **O golpe carimbado precisa de Rigidbody2D?** Só se o inimigo não tiver. Se o golpe tiver rb, Body Type **Kinematic** (ou Gravity Scale 0) — senão o golpe **cai** pelo chão.
 - `Is Trigger` é do **collider**, não do **Rigidbody**. É uma caixinha no `BoxCollider2D`.
 - Trigger dispara uma vez por objeto que entra. Objeto que fica parado dentro → só `Stay`.
 - Trigger não faz o boneco "pisar": chão trigger = boneco cai através. Chão é collider sólido.

Formas de usar

```
// 1) Moeda: some ao ser tocada pelo player
public class Moeda : MonoBehaviour
{
    void OnTriggerEnter2D(Collider2D outro)
    {
        if (outro.CompareTag("Player"))
        {
            Debug.Log("moeda!");
            Destroy(gameObject);
        }
    }
}
```

```
// 2) Zona de morte no fundo da fase (um BoxCollider2D largo, Is Trigger, tag "Morte")
public class ZonaDeMorte : MonoBehaviour
{
    void OnTriggerEnter2D(Collider2D outro)
    {
        if (outro.CompareTag("Player")) outro.GetComponent<VidaPlayer>().Morrer();
    }
}
```

```
// 3) Área que machuca ENQUANTO está dentro (lava): Stay + timer
private float tempoDesdeDano = 0f;
void OnTriggerStay2D(Collider2D outro)
{
    tempoDesdeDano += Time.deltaTime;
    if (outro.CompareTag("Player") && tempoDesdeDano >= 1f)
    {
        outro.GetComponent<VidaPlayer>().TomarDano(1);
        tempoDesdeDano = 0f;
    }
}
```

O golpe da espada completo (`Espada.cs`) está na Parte 5.6.

3.5 Ground check profissional — `Physics2D.OverlapCircle` e `Raycast` [peça em uso, aula de 04/09]

(04/09/2026 — substituiu o ground check por trombada em `aula_8/Assets/Scripts/Movimento.cs`)

* Simples assim

Em vez de esperar a campainha da trombada (que toca pra qualquer lado, até pra parede), o boneco ganha um "dedão" invisível na sola do pé: um círculo pequeno, colado num objeto vazio filho (`checadorDeChao`). Todo quadro eu pergunto: "esse círculo está encostando em algo da turma **Chão?**" Se sim, estou no chão; se não, estou no ar. É uma pergunta feita de novo a cada quadro — então cair da beirada, encostar na parede ou pisar numa plataforma móvel dão a resposta certa sozinhos.

- **O problema (o boneco preso na parede):** o `OnCollisionEnter2D` toca pra QUALQUER lado do collider — inclusive as laterais. Encostei na parede no meio do pulo? O alarme tocou, `estaNoChao` virou `true`, e o boneco pulava de novo grudado na parede (ou "subia" a parede a cada aperto). O alarme não sabe ONDE encostou, só que encostou. Somado ao furo da beirada (cair sem pular deixava o chão ligado), o ground check por trombada não segurava.
- **A solução:** perguntar em vez de esperar: `Physics2D.OverlapCircle(centro, raio, camada)` posicionado **exatamente nos pés**. Ele devolve o collider que está dentro do círculo (ou `null`), e `null` vira `false` num `bool`.
- **Como funciona:** um círculo invisível numa área específica (`checadorDeChao`, um objeto vazio filho do Boneco, na sola do pé), conferindo **só a camada certa** (`LayerMask camadaChao`). Se o círculo encosta em algo da layer do chão, o boneco está no solo. Como a resposta é recalculada a cada quadro, cair da beirada, encostar na parede ou pisar em plataforma móvel dão a resposta certa sem código extra.

A receita no editor [aula de 04/09]

1. **Criar o checador:** Hierarchy → botão direito no **Boneco** → *Create Empty* → renomear pra ChecadorDeChao (é filho: fica indentado embaixo do Boneco e vira junto com ele).
2. **Posicionar:** com o ChecadorDeChao selecionado, ferramenta *Move (W)* e arrastar até a **base do collider**, um tiquinho abaixo da sola. O círculo tem que encostar no chão quando o boneco está pisando — e só aí. Conferir na Scene com o Boneco selecionado: a casca verde do collider mostra onde é a sola.
3. **Layer do chão:** *Layer* (topo do Inspector) → *Add Layer...* → escrever Chao numa linha vazia → voltar em CADA objeto de chão/plataforma → *Layer* → Chao. O Boneco fica em *Player* (nunca em Chao).
4. **No script (topo da classe):** `public Transform checadorDeChao;` · `public float raioChecador = 0.2f;` · `public LayerMask camadaChao;`
5. **No Inspector do Boneco:** arrastar o ChecadorDeChao da Hierarchy pra caixinha checadorDeChao; em camadaChao marcar **só** Chao.
6. **Testar:** Play → selecionar o Boneco → o gizmo amarelo mostra o círculo na Scene. Pulando, ele sai do chão; parado, encosta. Encostado na parede no ar, NÃO pode encostar em nada — se encosta, o raio está grande ou o checador está alto.

```
[Header("Chão")]
public Transform checadorDeChao;           // objeto vazio filho, na sola do pé
public float raioChecador = 0.2f;         // tamanho do círculo
public LayerMask camadaChao;              // só o que está nessa camada conta como chão
private bool estaNoChao;                  // a resposta, recalculada todo quadro

void Update()
{
    // pergunta todo quadro: tem chão dentro do círculo? (Collider2D ou null → vira bool)
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao);

    if (Input.GetButtonDown("Jump") && estaNoChao)
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo);
}

// Desenha o círculo na Scene quando o Boneco está selecionado – pra ajustar raio e posição
void OnDrawGizmosSelected()
{
    if (checadorDeChao == null) return;
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(checadorDeChao.position, raioChecador);
}
```

Mais fundo: o antigo × o novo

	<code>OnCollisionEnter2D</code> (até 03/09)	<code>Physics2D.OverlapCircle</code> (desde 04/09)
quando responde	só no INSTANTE do toque (alarme)	todo quadro (pergunta)
sabe de que lado encostou?	não — parede lateral também liga	sim — o círculo está só nos pés
cair da beirada sem pular	fica ligado (furo)	desliga sozinho no quadro seguinte
plataforma móvel, rampa	depende do Exit tocar	funciona

	<code>OnCollisionEnter2D</code> (até 03/09)	<code>Physics2D.OverlapCircle</code> (desde 04/09)
o que precisa	tag <code>Ground</code> no chão	layer <code>Chao</code> no chão + objeto vazio no pé + <code>LayerMask</code> marcada
quem escreve em <code>estaNoChao</code>	Enter liga, o pulo desliga	só a linha do <code>Update</code> (1.11)

- **A versão com raio (Raycast):** dispara uma linha pra baixo e vê se acertou algo da camada. Boa pra medir distância até o chão (pouso, sombra).

```
RaycastHit2D acerto = Physics2D.Raycast(transform.position, Vector2.down, 1.1f, camadaChao);
estaNoChao = acerto.collider != null;
Debug.DrawRay(transform.position, Vector2.down * 1.1f, Color.red); // pra ver na Scene
```

- **Regras / pegadinhas:**

- `OnCollisionEnter2D` + `estaNoChao = false` no pulo **foram apagados em 04/09** — o `OverlapCircle` substitui o ground check inteiro. Não misturar os dois (um liga, o outro desliga, vira briga — 1.11).
- Raio pequeno demais (0.02) falha em rampa; grande demais (0.5) liga encostado na parede. 0.1 a 0.25 é o normal. Ajustar olhando o gizmo amarelo.
- Se o **player** estiver na layer `Chao`, o círculo acha o próprio collider do player e `estaNoChao` fica ligado pra sempre. Player em `Player`, chão em `Chao`.
- `LayerMask` vazia = nunca acha. Caixa `checadorDeChao` vazia = `NullReferenceException` na primeira linha do `Update`. Conferir as duas caixinhas.
- O checador é filho: quando o boneco vira (`localScale.x = -1`), ele espelha junto. Por isso fica no MEIO da sola ($x = 0$), não num canto — senão troca de lado a cada virada.
- Sprite com pivô *Bottom* (4.6): o `transform.position` do Boneco passa a ser a sola, e o checador fica em $y \approx 0$ (um tiquinho negativo). Com pivô *Center*, fica em $y \approx$ menos metade da altura.
- Mesma ideia serve pra **parede** (círculo na frente, na altura do peito → wall jump) e pra **teto** (círculo em cima → não pode levantar do agachado).

PARTE 4 — MOVIMENTO DO PERSONAGEM, PEÇA POR PEÇA

4.1 rb.linearVelocity também se Lê — pulo com altura controlada

(03/09/2026 — peça do toque=pulinho / segurar=pulão, em `Movimento.cs`)

* Simples assim

A velocidade não é só pra escrever: ela também conta o que o boneco está fazendo. y positivo = subindo, y negativo = caindo, zero = parado na vertical. Com isso o corte do pulo fica fácil: soltou o botão enquanto ainda sobe? Diminui a subida e o pulo fica mais baixo. É o mesmo número que a animação de pulo/queda lê (7.6).

- **O que é:** até aqui eu só ESCREVIA na velocidade. Mas ela se lê como qualquer número, e conta a história do boneco a cada quadro:
- `rb.linearVelocity.y > 0` → subindo
- `rb.linearVelocity.y < 0` → caindo
- **Pra que serve:** o pulo de metroidvania — no instante em que o botão é SOLTO (`Input.GetButtonUp("Jump")`, o Up da família), se ainda estiver subindo, **encolher** a velocidade de subida (multiplicar por uma fração tipo `0.5f`). A subida perde força, o pulo morre mais baixo, a gravidade faz o resto. Solto cedo = pulinho; segurou = pulão.
- **Regras / pegadinhas:**
 - O guarda `> 0` é obrigatório: sem ele o corte dispara também na QUEDA, e fração de número negativo = queda mais lenta = boneco-pena flutuando.
 - A escrita continua preservando o eixo alheio (x fica como está) — terceira irmã das linhas 18 e 21.
 - Fração no código é número mágico — campo `public float` bota ela no Inspector.

Mais fundo: outras coisas que a velocidade conta

```
bool subindo    = rb.linearVelocity.y > 0.01f;
bool caindo     = rb.linearVelocity.y < -0.01f;
bool andando    = Mathf.Abs(rb.linearVelocity.x) > 0.01f;
bool parado     = rb.linearVelocity == Vector2.zero;
float rapidez    = rb.linearVelocity.magnitude;           // sem direção, só o "quanto"
```

Tudo isso alimenta o Animator (7.6): hoje são `subindo / caindo` + `estaNoChao` que ligam os interruptores `IsJump` e `IsFall`; a versão com `SetFloat("velY", ...)` é peça futura.

4.2 Eixos: x é pro LADO, y é pra CIMA

(03/09/2026 — dúvida minha respondida)

* **Simples assim**

x é a régua deitada (esquerda/direita), y é a régua em pé (cima/baixo). `new Vector2(x, y)` : primeiro o lado, depois a altura — sempre nessa ordem. Pra decorar: o "y" tem um pezinho e fica em pé; o "x" fica deitado.

- **x = horizontal** (deitado: esquerda/direita) · **y = vertical** (em pé: cima/baixo). Mesmo desenho do gráfico da escola.
- `Vector2(x, y)` : **a ordem é sempre primeiro o lado, depois a altura.**
- **No meu código:** linha 18, o andar preenche a 1ª casinha (`velocidade * lado, x`); linha 21, o pulo preenche a 2ª (`forçaPulo, y`).

Complemento: **z é a profundidade** (pra dentro da tela). Em 2D o z só importa pra ordem de desenho e pra câmera (z = -10). Sprites em z = 0; quem precisa ficar na frente usa *Order in Layer* do `SpriteRenderer`, não o z.

4.3 O pulo completo — a versão de referência [peça consolidada]

* **Simples assim**

O pulo tem três pedaços: (1) apertou e está no chão → joga o boneco pra cima; (2) soltou cedo → diminui a subida (pulinho); (3) o círculo nos pés diz, todo quadro, se está no chão. Juntando os três, é o pulo de plataforma completo. Se um pedaço falha, cada um tem a sua seção: 3.5 pro chão, 4.1 pro corte.

O que eu tenho hoje, 04/09 (apelido no pulo, corte no soltar, ground check por círculo nos pés) junto num bloco:

```
[Header("Pulo")]
public float forçaPulo = 12f;
[Range(0f, 1f)] public float corteDoPulo = 0.5f;

[Header("Chão")]
public Transform chegadorDeChao; // objeto vazio na sola do pé (3.5)
public float raioChegador = 0.2f;
public LayerMask camadaChao;

private bool estaNoChao;

void Update()
{
    // 0) o círculo nos pés responde TODO quadro: estou no chão? (único dono da caixa - 1.11)
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChegador, camadaChao);

    // 1) apertou e está no chão → pula
    if (Input.GetButtonDown("Jump") && estaNoChao)
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, forçaPulo); // irmã 2
        // sem "estaNoChao = false" aqui: no próximo quadro o círculo já responde "no ar" sozinho
    }

    // 2) soltou cedo, ainda subindo → corta
    if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0f)
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * corteDoPulo); //
irmã 3
    }
}
```

```
void OnDrawGizmosSelected()    // o círculo amarelo na Scene, pra ajustar (3.5)
{
    if (checadorDeChao == null) return;
    Gizmos.color = Color.yellow;
    Gizmos.DrawWireSphere(checadorDeChao.position, raioChecador);
}
```

O que SAIU em relação à versão de 03/09: o `OnCollisionEnter2D` que ligava `estaNoChao` (contava parede como chão) e o `estaNoChao = false` dentro do pulo (agora o círculo decide). Os dois furos de 03/09 — beirada e parede — fecharam com isso.

Números de partida que costumam ficar bons: Gravity Scale **3**, `forçaPulo` **12**, `velocidade` **6**, `corteDoPulo` **0.5**. Ajustar no Inspector com o jogo rodando e **anotar**.

4.4 Coyote time e jump buffer — o pulo que perdoa [peça futura]

* Simples assim

São dois "perdões" de um décimo de segundo: apertou pular um pouquinho DEPOIS de sair da beirada? Vale. Apertou um pouquinho ANTES de aterrissar? Também vale. O jogador nem percebe, mas para de jurar que "apertei e não pulou". Cada perdão é só um número que desce com o tempo (6.1).

- **O que é:** dois timers pequenos que fazem o pulo parecer "justo":
- **Coyote time:** depois de sair da beirada, ainda dá pra pular por ~0,1 s (o coioote do desenho que corre no ar antes de cair). O jogador aperta um tiquinho tarde e o jogo perdoa.
- **Jump buffer:** apertar Jump um tiquinho ANTES de aterrissar guarda o pedido por ~0,1 s e pula assim que tocar o chão.
- **Pra que serve:** sem eles, o jogador jura que apertou e o boneco não pulou. Com eles, o controle fica "gostoso" — é a diferença mais barata entre jogo amador e jogo bom.
- **A receita:** dois números que descem com `deltaTime`; o chão recarrega um, o aperto recarrega o outro; o pulo acontece quando os dois estão acima de zero.

```
[Header("Perdão do pulo")]
public float tempoCoyote = 0.1f;
public float tempoBuffer = 0.1f;
private float coyote;    // quanto tempo ainda posso pular depois de sair do chão
private float buffer;    // quanto tempo ainda vale o aperto que fiz

void Update()
{
    // ground check (3.5)
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao);

    // recarrega ou desce
    if (estaNoChao) coyote = tempoCoyote; else coyote -= Time.deltaTime;
    if (Input.GetButtonDown("Jump")) buffer = tempoBuffer; else buffer -= Time.deltaTime;

    // pula se os dois ainda valem
    if (buffer > 0f && coyote > 0f)
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, forçaPulo);
        buffer = 0f;    // gasta o pedido
        coyote = 0f;    // gasta o perdão (senão pula 2x na beirada)
    }
}
```

```

    if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0f)
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * corteDoPulo);
}

```

- **Pegadinhas:** esquecer de zerar `coyote` no pulo = pulo duplo grátis na beirada. Timer de 0.3 s ou mais deixa de ser perdão e vira pulo no ar visível — 0.08 a 0.15 é a faixa.

4.5 Dash de verdade — velocidade + tempo [peça futura]

* Simples assim

O dash de verdade é "por 0,15 segundo a velocidade é enorme e a gravidade some". Como é velocidade (e não teleporte), a parede segura. Enquanto o dash dura, o andar normal não pode escrever na velocidade — senão apaga o dash no quadro seguinte. É pra isso que existe o `return` no meio do `Update`.

- **O que é:** o dash v1 teleportou (`localPosition`, atravessa parede). O de verdade **crava uma velocidade alta** por um **tempo curto**, com a gravidade desligada, e depois devolve o controle. Como é velocidade, o motor de física respeita paredes.
- **As caixinhas:** `velocidadeDash` (20 a 30), `duracaoDash` (0.15 s), `recargaDash` (0.5 s).
- **O estado:** enquanto `dando` for `true`, o andar normal NÃO crava a velocidade (senão apaga o dash no quadro seguinte — pegadinha do 1.7).

```

[Header("Dash")]
public float velocidadeDash = 25f;
public float duracaoDash    = 0.15f;
public float recargaDash    = 0.5f;
private bool dando;
private float tempoDash;    // quanto falta do dash
private float recarga;      // quanto falta pra poder de novo
private float gravidadeNormal;

void Start()
{
    gravidadeNormal = rb.gravityScale;
}

void Update()
{
    recarga -= Time.deltaTime;

    // começar
    if (Input.GetButtonDown("Dash") && recarga <= 0f && !dando)
    {
        dando      = true;
        tempoDash  = duracaoDash;
        recarga     = recargaDash;
        rb.gravityScale = 0f; // dash reto, sem cair
    }

    // durante
    if (dando)
    {
        rb.linearVelocity = new Vector2(direcao * velocidadeDash, 0f); // pra onde olho, sem y
        tempoDash -= Time.deltaTime;
        if (tempoDash <= 0f)
        {
            dando = false;
            rb.gravityScale = gravidadeNormal; // devolve a gravidade
        }
        return; // pula o resto do Update
    }
}

```

```
(andar/ pular)
}

// andar normal só roda quando NÃO está dando dash
float lado = Input.GetAxisRaw("Horizontal");
rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y);
}
```

• Regras / pegadinhas:

- O `return` no meio do `Update` é o truque: enquanto dá dash, nada abaixo roda.
- Esquecer de devolver `gravityScale` = boneco flutua pra sempre depois do primeiro dash.
- `direcao` tem que ser +1/-1 e estar atualizada (4.7) — dash pra direção zero = boneco para.
- A deslizada (S + Dash) da minha receita de combo entra aqui: mesmo bloco, com o agachar ligado.
- Versão com corrotina (mais limpa) na Parte 6.3.

4.6 Agachar — `localScale` hoje, `collider` amanhã [peça em uso + futura]

* Simples assim

Hoje o agachar encolhe a escala (o boneco "achata"). Funciona, mas tem dois detalhes: o desenho fica espremido, e se o pivô do sprite estiver no meio, o boneco encolhe pros dois lados e fica com os pés no ar — agacha "flutuando". A versão profissional troca o sprite e encolhe só o collider. A correção do pivô (aula de 04/09) está logo abaixo.

- **Hoje (minhas linhas 29-33 e 51-52):** `GetKeyDown(KeyCode.S)` liga `pequeno`, `GetKeyUp` desliga; o ternário escolhe `escalaAgachado` ou `escalaBase` pro `y` do `localScale`. Funciona e encolhe o collider junto.
- **O problema:** o sprite fica espremido (o desenho achata). O jeito profissional é **trocar o sprite** (ou animação) e **encolher só o collider**.

```
[Header("Agachar")]
public BoxCollider2D corpo; // arrastar o BoxCollider2D do Boneco
public Vector2 tamanhoEmPe = new Vector2(0.8f, 1.8f);
public Vector2 tamanhoAgachado = new Vector2(0.8f, 1.0f);
public Vector2 centroEmPe = new Vector2(0f, 0f);
public Vector2 centroAgachado = new Vector2(0f, -0.4f); // desce o centro pra base ficar no chão
public Transform cabeca; // objeto vazio no topo, pra checar teto
public float raioCabeca = 0.2f;
private bool agachado;

void Update()
{
    if (Input.GetKeyDown(KeyCode.S)) Agachar(true);

    // só levanta se NÃO tem teto em cima
    if (Input.GetKeyUp(KeyCode.S) && !Physics2D.OverlapCircle(cabeca.position, raioCabeca, camadaChao))
        Agachar(false);
}

void Agachar(bool sim)
{
    agachado = sim;
    corpo.size = sim ? tamanhoAgachado : tamanhoEmPe;
}
```

```
corpo.offset= sim ? centroAgachado : centroEmPe;
}
```

- **Pegadinhas:** mudar `size` sem mudar `offset` encolhe pelo centro e o boneco "afunda" ou "flutua"; se tem teto e o jogador solta o S, ele continua agachado — precisa checar de novo a cada quadro enquanto `agachado && !Input.GetKey(KeyCode.S)`; velocidade de andar agachado costuma ser menor (`velocidade * 0.5f`).

O agachar que flutua — o pivô do sprite [aula de 04/09]

- **O sintoma:** apertei S e o boneco encolheu — mas os pés subiram: ele "agachou no ar", e a gravidade puxou ele de volta pro chão (um pulinho pra baixo). Ao soltar o S, cresceu pra dentro do chão e a física empurrou pra cima. Um tremelique a cada agachar.
- **A causa:** `transform.localScale` encolhe o objeto **em volta do pivô** do sprite. Com o pivô no **Center** (o padrão), `y = 0.5` tira metade de cima e metade de baixo — a sola sobe. O collider encolhe do mesmo jeito (o centro dele continua no mesmo lugar).
- **A solução:** mudar o pivô do sprite pra **Bottom**. Aí a escala encolhe só de cima pra baixo: a sola fica onde está e o boneco agacha com os pés no chão.

Trocar o pivô pra Bottom

1. Project → clicar no arquivo do sprite (o `.png`) → Inspector.
2. *Sprite Mode* **Single**: campo **Pivot** → Bottom → **Apply** (botão no rodapé do Inspector). *Sprite Mode Multiple* (sprite sheet): botão **Sprite Editor** → selecionar cada quadro (ou todos) → *Pivot* → Bottom → **Apply** no topo da janela.
3. Conferir na Scene: o pontinho azul do pivô fica na sola do desenho.
4. **Reajustar o collider:** o desenho subiu em relação ao `transform` (a posição do objeto agora é a sola). *Edit Collider* no *BoxCollider2D* e arrastar até a casca verde abraçar o desenho de novo (o *Offset y* fica $\approx$ metade da altura).
5. Conferir o filho *ChecadorDeChao* (3.5): ele continua em relação ao `transform`, que agora é a sola — mover pra `y  $\approx$  0` (um tiquinho negativo).

pivô	<code>localScale.y = 0.5</code> faz o quê	sensação
Center (padrão)	encolhe metade em cima, metade embaixo	pés sobem: "agacha no ar" e cai; ao levantar, afunda e é empurrado
Bottom	encolhe só de cima pra baixo	pés parados no chão — o agachar certo
Top	encolhe só de baixo pra cima	pés sobem inteiros (bom pra estalactite, não pra boneco)

- **Pegadinhas:** mudar o pivô muda onde `transform.position` "é" — golpe nascendo em `transform.position` passa a nascer na sola (usar o `pontoDoGolpe`, 1.8); prefab com o sprite antigo continua com o pivô antigo até trocar o asset; `Instantiate` com um sprite de pivô Bottom nasce com a sola no ponto pedido.

4.7 Virar o boneco — direcao +.localScale x flipX [peça em uso]

* Simples assim

Virar o boneco é multiplicar a escala x por -1: o desenho vira de lado como uma folha de papel. `direcao` guarda o último lado olhado (+1 ou -1) pra ele não "esquecer" pra onde olha quando solto a tecla. Sem esse cuidado a direção vira 0, a escala vira 0 e o boneco some.

jeito	como	espelha os filhos?	quando
<code>localScale.x = direcao * escalaBase</code> (o meu, linha 52)	inverte a escala inteira	sim — <code>PontoDoGolpe</code> , <code>ChecadorDeChao</code> , tudo vira junto	quase sempre (Fase 2 precisa disso)
<code>spriteRenderer.flipX = direcao < 0</code>	espelha só o desenho	não	objeto sem filhos, animação simples

```
// A regra que guarda a última direção olhada (parado continua olhando pro mesmo lado)
float lado = Input.GetAxisRaw("Horizontal");
if (lado != 0f) direcao = Mathf.Sign(lado); // +1 ou -1, nunca 0

transform.localScale = new Vector3(direcao * escalaBase, escalaY, 1f);
```

- **Pegadinhas:** `direcao = lado` sem o `if` zera a direção quando solto a tecla → escala 0 → boneco some. `escalaBase` guarda o tamanho original (ex.: 1 ou 0.5) pra não crescer/encolher a cada virada.

4.8 Movimento.cs de referência — tudo junto [código de referência, não o meu arquivo]

* Simples assim

Este é o script inteiro, com todas as peças juntas, pra comparar com o meu — não pra copiar por cima. Leitura de cima pra baixo: caixinhas do Inspector, gavetas internas, `Awake` (se arruma), `Update` (todo quadro: chão, andar, virar, pular, dash, animação). Cada bloco tem a sua seção no manual, indicada no comentário.

Um controlador de plataforma completo com as peças deste manual (ground check por círculo nos pés, pulo com corte, coyote/buffer, virar, dash, animação de pulo/queda). Pra comparar com o meu, não pra substituir. Atualizado em 04/09: sem alarme de colisão pro chão, com o `Animar()` no fim do `Update`.

```
using UnityEngine;

public class Movimento : MonoBehaviour
{
    [Header("Andar")]
    public float velocidade = 6f;

    [Header("Pulo")]
    public float forcaPulo = 12f;
    [Range(0f, 1f)] public float corteDoPulo = 0.5f;
    public float tempoCoyote = 0.1f;
    public float tempoBuffer = 0.1f;

    [Header("Dash")]
    public float velocidadeDash = 25f;
    public float duracaoDash = 0.15f;
    public float recargaDash = 0.5f;
```

```

[Header("Chão")]
public Transform checadorDeChao; // objeto vazio na sola do pé (3.5)
public float raioChecador = 0.2f;
public LayerMask camadaChao;

[Header("Animação")]
public Animator animator; // IsJump / IsFall (7.6)

[Header("Referências")]
public Rigidbody2D rb;

[HideInInspector] public float direcao = 1f; // porta pro Ataque.cs, sem poluir o Inspector
[HideInInspector] public bool estaNoChao;

private float escalaBase;
private float coyote, buffer;
private bool dando;
private float tempoDash, recarga, gravidadeNormal;

void Awake()
{
    if (rb == null) rb = GetComponent<Rigidbody2D>(); // se esqueci de arrastar, acho sozinho
    escalaBase = Mathf.Abs(transform.localScale.x);
    gravidadeNormal = rb.gravityScale;
}

void Update()
{
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao); //
3.5 (1.11)
    recarga -= Time.deltaTime;

    if (dando) { Dashando(); Animar(); return; }

    float lado = Input.GetAxisRaw("Horizontal");
    Andar(lado);
    Virar(lado);
    Pular();

    if (Input.GetButtonDown("Dash") && recarga <= 0f) ComecarDash();

    Animar(); // 7.6 – por último, com a velocidade deste
quadro
}

void Andar(float lado)
{
    rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y);
}

void Virar(float lado)
{
    if (lado != 0f) direcao = Mathf.Sign(lado);
    transform.localScale = new Vector3(direcao * escalaBase, transform.localScale.y, 1f);
}

void Pular()
{
    if (estaNoChao) coyote = tempoCoyote; else coyote -= Time.deltaTime;
    if (Input.GetButtonDown("Jump")) buffer = tempoBuffer; else buffer -= Time.deltaTime;

    if (buffer > 0f && coyote > 0f)
    {
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo);
        buffer = 0f;
        coyote = 0f;
    }

    if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0f)

```

```

        rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * corteDoPulo);
    }

    void ComecarDash()
    {
        dando = true;
        tempoDash = duracaoDash;
        recarga = recargaDash;
        rb.gravityScale = 0f;
    }

    void Dashando()
    {
        rb.linearVelocity = new Vector2(direcao * velocidadeDash, 0f);
        tempoDash -= Time.deltaTime;
        if (tempoDash <= 0f)
        {
            dando = false;
            rb.gravityScale = gravidadeNormal;
        }
    }

    void Animar()
    {
        if (!estaNoChao && rb.linearVelocity.y > 0.1f)           // subindo no ar
        {
            animator.SetBool("IsJump", true);
            animator.SetBool("IsFall", false);
        }
        else if (!estaNoChao && rb.linearVelocity.y < -0.1f)    // caindo no ar
        {
            animator.SetBool("IsJump", false);
            animator.SetBool("IsFall", true);
        }
        else                                                     // no chão
        {
            animator.SetBool("IsJump", false);
            animator.SetBool("IsFall", false);
        }
    }

    void OnDrawGizmosSelected()
    {
        if (checadorDeChao == null) return;
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(checadorDeChao.position, raioChecador);
    }
}

```

Checklist pra esse script rodar em casa: Boneco com `Rigidbody2D` (Gravity Scale 3, Freeze Rotation Z), `BoxCollider2D` com o material `SemAtrito` (3.1), filho `ChecadorDeChao` na sola arrastado na caixinha (3.5), layer do chão = `Chao` marcada em `camadaChao`, sprite com pivô *Bottom* (4.6), `Animator` com os bools `IsJump` / `IsFall` arrastado na caixinha (7.6), apelido `"Dash"` criado no Input Manager (2.6), tag `Player` no Boneco.

PARTE 5 — FASE 2: COMBATE EM TEMPO REAL

5.1 public GameObject — caixinha de Inspector que guarda um OBJETO inteiro

(03/09/2026 — peça 1 da Fase 2: o ouvido do ataque, `ataques/Ataque.cs`)

* Simples assim

A caixinha do Inspector não guarda só número: com o tipo `GameObject` ela guarda um objeto inteiro (ou um molde/prefab). Arrasto o prefab do golpe pra dentro e o código passa a ter o molde na mão pra carimbar. Caixa vazia = `NullReferenceException` na hora de usar.

- **O que é:** campo `public` não é só pra número: o tipo `GameObject` faz a caixinha do Inspector aceitar um **objeto/molde inteiro** — arrasta lá, igual fiz com o `Rigidbody` na caixinha `rb`.
- **Pra que serve:** dar ao código o MOLDE do golpe (prefab) na mão, pra carimbar cópias com `Instantiate` (peça 3).
- **Regras / pegadinhas:**
 - Caixa vazia (esqueci de arrastar) = `NullReferenceException` na hora de usar — a Unity aponta a linha.
 - Quem escuta o botão de ataque é quem está SEMPRE vivo em cena (o player); a espada não pode esperar o próprio nascimento pra ouvir o teclado. Player escuta e manda nascer; o script do objeto nascido cuida do dano.

Mais fundo: `GameObject` × `Transform` × componente na caixinha

tipo da caixinha	o que arrasto	o que ganho no código
<code>public GameObject golpePrefab;</code>	o objeto/prefab inteiro	<code>Instantiate(golpePrefab, ...)</code> , <code>.SetActive()</code> , <code>.GetComponent<></code>
<code>public Transform pontoDoGolpe;</code>	um objeto (a Unity pega o <code>Transform</code> dele)	<code>.position</code> pra nascer ali — sem precisar do objeto inteiro
<code>public Rigidbody2D rb;</code>	um objeto (a Unity pega o <code>Rigidbody2D</code> dele)	direto o componente
<code>public Movimento movimento;</code>	um objeto que tem o script	direto o meu script (5.3)

Regra: pedir na caixinha **exatamente o que vou usar**. Se só preciso da posição, `Transform`. Se vou carimbar, `GameObject`. Se vou ler um campo do script, o tipo do script.

5.2 `Instantiate(molde, onde, rotação)` — a máquina de carimbar

(03/09/2026 — peça 3 da Fase 2, em `ataques/Ataque.cs`)

* Simples assim

`Instantiate(molde, onde, rotação)` é o carimbo: pega o prefab e faz nascer uma cópia viva na cena, naquele lugar, agora. Cada carimbo é uma cópia nova (`Clone`) na Hierarchy), e ela fica lá até alguém apagar com `Destroy`.

- **O que é:** entrega um prefab, um lugar e uma rotação → nasce uma CÓPIA viva na cena naquele instante. É assim que nasce tudo em tempo de jogo: balas, moedas, inimigos, golpes.
- **Os 3 ingredientes em ordem:** 1. molde — o que está na caixinha (`golpePrefab`) 2. onde — `transform.position` = a posição de QUEM FALA (crachá do dono do script, mesma família do `transform.localScale`) 3. rotação — `Quaternion.identity` = "reto, do jeito que o molde é" (decorar como carimbo-sem-girar por enquanto)
- **Regras / pegadinhas:**
 - Caixinha `golpePrefab` vazia no Inspector = `NullReferenceException` na linha do `Instantiate`.
 - Carimbo NÃO tem borracha: cada cópia fica na cena até alguém `Destroy` (peça 4). Hierarchy enchendo de (`Clone`) = sintoma esperado da v1.
 - Nascer NA FRENTE (com direção) = peça 3.5, exige dois scripts conversando.

Mais fundo: as formas do carimbo

```
Instantiate(golpePrefab); // no (0,0,0), rotação do molde
Instantiate(golpePrefab, transform.position, Quaternion.identity); // a que uso
Instantiate(golpePrefab, transform.position, Quaternion.identity, transform); // já nasce FILHO do boneco
Instantiate(golpePrefab, pontoDoGolpe.position, Quaternion.identity); // na posição do filho vazio (1.8)
Instantiate(golpePrefab, transform.position, Quaternion.Euler(0f, 0f, 45f)); // girado 45°
GameObject copia = Instantiate(golpePrefab, transform.position, Quaternion.identity); // guardar a cópia
```

- A cópia vem com **todos os componentes e scripts do molde**, e o `Awake` / `Start` dela rodam na hora do carimbo — é ali que a espada se prepara.
- Carimbar uma cópia da CENA (não prefab) também funciona (`Instantiate(gameObject)` clona a si mesmo) — raro, mas explica o "(Clone)(Clone)".
- Carimbo dentro do `Update` sem guarda = 60 golpes por segundo. Sempre atrás de um `GetButtonDown` ou de um timer.

5.3 Dois scripts conversando — caixinha de vizinho + porta public

(03/09/2026 — peça 3.5: o golpe nasce NA FRENTE)

* Simples assim

Um script pode ter uma caixinha cujo tipo é OUTRO script (`public Movimento movimento;`). Arrasto o boneco nela e pronto: `movimento.direcao` lê a variável do vizinho. Só funciona se a variável do vizinho for `public` — a porta aberta. `private` é gaveta trancada.

- **O que é:** um script meu é um TIPO como `Rigidbody2D`: `public Movimento movimento;` no `Ataque.cs` cria uma caixinha que segura o script do vizinho — arrasta o Boneco nela no Inspector (a Unity acha o componente). Depois, `movimento.direcao` lê o campo dele com coleira normal.

- **A porta:** vizinho só lê campo `public float direcao` é gaveta trancada → virar `public float direcao` pro Ataque enxergar.
- **A conta do "na frente"** (a MESMA fórmula do meu dash, linha 34): `transform.position.x + distanciaGolpe * movimento.direcao` no x, y intacto, 0 no z.
- **Pegadinha:** caixinha `movimento` vazia no Inspector = `NullReference` no aperto, igual à do prefab.

Mais fundo: os 3 jeitos de um script chegar no outro

jeito	quando o vizinho é...	código
caixinha (o meu)	outro objeto da cena, fixo, conhecido no editor	<code>public Movimento movimento;</code> + arrastar
<code>GetComponent</code> (5.4)	o MESMO objeto, ou um objeto que acabei de tocar/carimbar	<code>GetComponent<Movimento>()</code> , <code>outro.GetComponent<Inimigo>()</code>
procurar na cena	um objeto único que o prefab não conhece (o player)	<code>GameObject.FindWithTag("Player")</code> , <code>FindFirstObjectByType<Movimento>()</code>

Os dois scripts no MESMO objeto (Ataque.cs e Movimento.cs, ambos no Boneco) é o caso perfeito do `GetComponent` no `Awake`: `movimento = GetComponent<Movimento>();` — nunca mais esqueço de arrastar.

5.4 GetComponent<T>() — achar um componente sem arrastar [peça futura, chave]

* Simples assim

`GetComponent<Rigidbody2D>()` é "me dá o `Rigidbody2D` que está neste objeto" — o jeito de achar um componente sem arrastar no Inspector. Com um dono na frente (`outro.GetComponent<Inimigo>()`) procura no outro objeto. Se não achar, devolve `null` e a linha seguinte explode — por isso sempre tem um `if` de guarda.

- **O que é:** pede a um `GameObject` "me dá o componente do tipo T". Sem coleira, procura em MIM (`GetComponent<Rigidbody2D>()`); com coleira, procura no outro (`outro.GetComponent<Inimigo>()`, `colisao.gameObject.GetComponent<Inimigo>()`).
- **Pra que serve:** é a ponte da Fase 2: a espada toca um collider (`Collider2D` `outro`) e precisa do **script** que está nele pra chamar `TomarDano`. Também substitui o arrastar do `rb`.
- **A família:**

chamada	procura em	devolve
<code>GetComponent<T>()</code>	neste objeto	o componente ou <code>null</code>
<code>outro.GetComponent<T>()</code>	no objeto <code>outro</code>	idem
<code>GetComponentInChildren<T>()</code>	neste objeto e nos filhos	o primeiro que achar
<code>GetComponentInParent<T>()</code>	neste objeto e nos pais	idem
<code>TryGetComponent(out T comp)</code>	neste objeto	<code>true</code> / <code>false</code> , e preenche <code>comp</code> — sem <code>null</code>
<code>GameObject.FindWithTag("Player")</code>	a cena inteira	o objeto, ou <code>null</code>

chamada	procura em	devolve
<code>FindFirstObjectByType<Movimento>()</code>	a cena inteira	o primeiro script desse tipo (Unity 6; antes era <code>FindObjectOfType</code>)

• Regras / pegadinhas:

- Se o objeto NÃO tem o componente, devolve `null` — e a linha seguinte explode com `NullReferenceException`. **Sempre guardar:** `if (inimigo != null)` ou `TryGetComponent`.
- `GetComponent` custa: fazer no `Awake` / `Start` e guardar numa gaveta, não todo quadro no `Update`. No alarme de trigger (uma vez por toque) pode.
- `FindWithTag` / `FindFirstObjectByType` varrem a cena inteira: só no `Start`, nunca no `Update`.
- O `<T>` entre sinais de menor/menor é o tipo — sem `new`, sem aspas: `GetComponent<Inimigo>()`.

Formas de usar

```
// 1) Pegar meu próprio Rigidbody2D (adeus, arrastar)
private Rigidbody2D rb;
void Awake() { rb = GetComponent<Rigidbody2D>(); }
```

```
// 2) Chamar o método do inimigo que acabei de tocar (com guarda)
void OnTriggerEnter2D(Collider2D outro)
{
    Inimigo inimigo = outro.GetComponent<Inimigo>();
    if (inimigo != null) inimigo.TomarDano(dano);
}

// 2b) O mesmo, com TryGetComponent (sem null na mão)
void OnTriggerEnter2D(Collider2D outro)
{
    if (outro.TryGetComponent(out Inimigo inimigo)) inimigo.TomarDano(dano);
}
```

```
// 3) A espada (prefab) acha o player na cena sem caixinha
private Transform player;
void Start() { player = GameObject.FindWithTag("Player").transform; }
```

5.5 Destroy — a borracha do carimbo [peça 4 do roadmap]

* Simples assim

`Destroy(gameObject)` apaga o objeto da cena. Com um número (`Destroy(gameObject, 0.15f)`) apaga daqui a 0,15 s — perfeito pro golpe que existe por um instante. Cuidado com o alvo: `Destroy(this)` apaga só o script e deixa o objeto sem cérebro.

- **O que é:** tira um objeto da cena. `Destroy(gameObject)` apaga o objeto onde o script está; `Destroy(outro.gameObject)` apaga o outro; `Destroy(gameObject, 0.2f)` **agenda** pra daqui a 0,2 s.
- **Pra que serve:** golpe que some sozinho, moeda pega, inimigo morto, bala que saiu da tela — a borracha do `(Clone)` que enchia a Hierarchy.
- **Regras / pegadinhas:**
 - `Destroy(this)` **apaga só o SCRIPT**, o objeto fica na cena sem cérebro. Pra apagar o objeto é `Destroy(gameObject)` — o `gameObject` minúsculo é o crachá do objeto dono do script.

- `Destroy(outro)` onde `outro` é `Collider2D` apaga só o **collider** do outro. Pra apagar ele inteiro: `Destroy(outro.gameObject)`.
- A destruição acontece no **fim do quadro**: o resto do método ainda roda. Depois disso, qualquer caixinha que apontava pra ele vira um "falso null" — `if (alvo == null)` funciona pra checar.
- Destruir o objeto onde uma corrotina roda mata a corrotina — normal, mas explica timers que "somem".
- Objeto que vai nascer e morrer o tempo todo (balas) — a versão profissional desliga e religa (`SetActive(false/true)`, *object pooling*) em vez de carimbar/apagar. Peça bem futura.

Formas de usar

```
Destroy(gameObject);           // eu mesmo, agora (fim do quadro)
Destroy(gameObject, 0.15f);    // eu mesmo, daqui a 0,15 s – o golpe da espada
Destroy(outro.gameObject);    // o outro objeto inteiro (moeda que o player pegou, visto pelo player)
gameObject.SetActive(false);    // NÃO destrói: esconde e desliga (dá pra religar depois)
```

```
// Bala que morre sozinha depois de 3 s, sem timer no Update
void Start() { Destroy(gameObject, 3f); }
```

5.6 Espada.cs — o golpe que machuca [peça 4, código de referência]

* Simples assim

É o script que vive no golpe: nasce, vira pro lado certo, agenda a própria morte e, se encostar num inimigo, chama `TomarDano`. A trava `jaBateu` garante que um golpe machuca uma vez só.

O molde `Golpe` (prefab): objeto vazio + `BoxCollider2D` com **Is Trigger** + `Rigidbody2D Kinematic` (pra garantir alarme mesmo em inimigo sem rb) + este script. Tag `Golpe`, layer `Golpe` (com `Golpe x Player` desmarcado na matriz — 3.3).

```
using UnityEngine;

public class Espada : MonoBehaviour
{
    public int dano = 1;
    public float duracao = 0.15f;           // quanto tempo o golpe existe
    public float empurrao = 6f;             // knockback no inimigo

    [HideInInspector] public float direcao = 1f; // quem carimba preenche (Ataque.cs)

    private bool jaBateu;                   // um golpe machuca UMA vez

    void Start()
    {
        // vira o golpe pro lado certo (o sprite/collider espelham junto)
        transform.localScale = new Vector3(direcao * Mathf.Abs(transform.localScale.x),
                                             transform.localScale.y, 1f);
        Destroy(gameObject, duracao);       // a borracha já fica agendada
    }

    void OnTriggerEnter2D(Collider2D outro)
    {
        if (jaBateu) return;
        if (!outro.CompareTag("Inimigo")) return;
    }
}
```

```

        if (outro.TryGetComponent(out Inimigo inimigo))
        {
            inimigo.TomarDano(dano, direcao * empurrao);
            jaBateu = true;
        }
    }
}

```

- **Por que** `jaBateu` : um golpe que fica 0,15 s pode gerar `Enter` em dois colliders do mesmo inimigo, ou acertar o inimigo que anda pra dentro dele duas vezes. Uma trava simples resolve.
- **Por que** `Start` **e não** `Awake` : o `Ataque.cs` preenche `direcao` **depois** do `Instantiate` e antes do primeiro quadro; o `Awake` roda DENTRO do `Instantiate`, cedo demais. O `Start` roda no quadro seguinte, já com a direção certa.

5.7 Inimigo.cs — vida, dano e morte [peça 5, código de referência]

* Simples assim

O inimigo é dono da própria vida: anda, vira na beirada e tem a porta `TomarDano` que qualquer coisa pode chamar (espada, bala, espinho). O timer `invencivel` dá um respiro entre um dano e outro e deixa o empurrão acontecer.

- **O que é:** o inimigo é quem VIVE — ele tem a vida e o método `TomarDano`, e qualquer coisa que machuca (espada, bala, armadilha) só chama esse método. A porta `public` no método é a regra do 5.3 aplicada a métodos.
- **Ingredientes do objeto:** `sprite`, `Rigidbody2D` (Dynamic, Freeze Rotation Z), `BoxCollider2D` sólido, `tag Inimigo`, `layer Inimigo`.

```

using System.Collections;           // pro IEnumerator do Piscar (Parte 6.3)
using UnityEngine;

public class Inimigo : MonoBehaviour
{
    [Header("Vida")]
    public int vida = 3;
    public float tempoInvencivel = 0.3f; // não toma dano de novo logo em seguida

    [Header("Patrulha")]
    public float velocidade = 2f;
    public Transform pe;                // filho vazio na frente/base, pra ver beirada
    public LayerMask camadaChao;

    private Rigidbody2D rb;
    private SpriteRenderer sprite;
    private float direcao = 1f;
    private float invencivel;           // timer

    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        sprite = GetComponent<SpriteRenderer>();
    }

    void Update()
    {
        invencivel -= Time.deltaTime;
        if (invencivel > 0f) return;    // machucado: sem andar (deixa o knockback agir)

        // vira na beirada: se NÃO tem chão na frente do pé, inverte
    }
}

```

```

    bool temChaoNaFrente = Physics2D.OverlapCircle(pe.position, 0.1f, camadaChao);
    if (!temChaoNaFrente) Virar();

    rb.linearVelocity = new Vector2(velocidade * direcao, rb.linearVelocity.y);
}

void OnCollisionEnter2D(Collision2D colisao)
{
    // bateu de lado numa parede → vira (normal apontando pro lado, não pra cima)
    if (Mathf.Abs(colisao.GetContact(0).normal.x) > 0.5f) Virar();
}

void Virar()
{
    direcao *= -1f;
    transform.localScale = new Vector3(direcao * Mathf.Abs(transform.localScale.x),
                                        transform.localScale.y, 1f);
}

// ---- a PORTA que a espada chama ----
public void TomarDano(int dano, float empurraoX)
{
    if (invencivel > 0f) return;

    vida -= dano;
    invencivel = tempoInvencivel;
    rb.linearVelocity = new Vector2(empurraoX, 4f);    // tranco pra trás e pra cima
    StartCoroutine(Piscar());                        // Parte 6.3

    if (vida <= 0) Morrer();
}

IEnumerator Piscar()
{
    sprite.color = Color.red;
    yield return new WaitForSeconds(0.1f);
    sprite.color = Color.white;
}

void Morrer()
{
    Destroy(gameObject);
}
}

```

• Regras / pegadinhas:

- `TomarDano` é `public` — sem isso a espada nem enxerga o método (CS0122 "inacessível").
- A assinatura tem que bater com a chamada: `TomarDano(int, float)` na espada e no inimigo. Mudou um lado, muda o outro (CS1501 "nenhuma sobrecarga aceita 1 argumento").
- O `return` no `Update` durante `invencivel` é o que deixa o knockback acontecer: sem ele o andar crava a velocidade no quadro seguinte e apaga o tranco (1.7).
- Inimigo × Inimigo desmarcado na Layer Collision Matrix — senão eles se empurram e "viram" um no outro.

5.8 Ataque.cs completo — ouvido, cooldown e carimbo com direção [código de referência]

* Simples assim

O player escuta o botão de ataque (Fire1), espera o cooldown e carimba o golpe na frente, passando a direção. Ele não machuca ninguém — quem machuca é o golpe (5.6). Cada script cuida de uma coisa.

```
using UnityEngine;

public class Ataque : MonoBehaviour
{
    [Header("Golpe")]
    public GameObject golpePrefab;           // peça 1: o molde
    public Transform pontoDoGolpe;          // filho vazio na frente do boneco (1.8) – espelha com a
    virada
    public float recargaAtaque = 0.35f;     // segundos entre golpes

    private Movimento movimento;            // peça 3.5, sem arrastar: mesmo objeto → GetComponent
    private float recarga;

    void Awake()
    {
        movimento = GetComponent<Movimento>();
    }

    void Update()
    {
        recarga -= Time.deltaTime;

        if (Input.GetButtonDown("Fire1") && recarga <= 0f)
        {
            Golpear();
            recarga = recargaAtaque;
        }
    }

    void Golpear()
    {
        // peça 3: carimbo no ponto da frente (o ponto já vira junto com o localScale do boneco)
        GameObject golpe = Instantiate(golpePrefab, pontoDoGolpe.position, Quaternion.identity);

        // peça 3.5: passa a direção pra cópia recém-nascida
        golpe.GetComponent<Espada>().direcao = movimento.direcao;
    }
}
```

- Sem o `pontoDoGolpe`, a conta da minha peça 3.5 continua valendo:
`new Vector3(transform.position.x + distanciaGolpe * movimento.direcao, transform.position.y, 0f).`
- **Ordem no Inspector do Boneco:** `Movimento` e `Ataque` no mesmo objeto; `golpePrefab` arrastado da PASTA (prefab, azul), nunca da cena; `pontoDoGolpe` arrastado da Hierarchy (é filho do Boneco).

5.9 Vida do player, dano de contato e recomeço [peça futura]

* Simples assim

Mesma ideia do inimigo, do lado do player: vida, invencibilidade que pisca, empurrão ao encostar no inimigo e recomeço da fase quando a vida zera. A pegadinha é o andar apagar o empurrão — o `Movimento` precisa saber que está machucado.

```
using UnityEngine;
using UnityEngine.SceneManagement; // pra recarregar a cena (Parte 7.2)

public class VidaPlayer : MonoBehaviour
{
    public int vidaMaxima = 3;
    public float tempoInvencivel = 1f;
    public float empurrao = 7f;

    private int vida;
    private float invencivel;
    private Rigidbody2D rb;
    private SpriteRenderer sprite;

    void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        sprite = GetComponent<SpriteRenderer>();
        vida = vidaMaxima;
    }

    void Update()
    {
        invencivel -= Time.deltaTime;
        // pisca enquanto invencível: liga/desliga o sprite a cada ~0,1 s
        if (invencivel > 0f) sprite.enabled = Mathf.FloorToInt(invencivel * 10f) % 2 == 0;
        else sprite.enabled = true;
    }

    void OnCollisionEnter2D(Collision2D colisao)
    {
        if (colisao.gameObject.CompareTag("Inimigo"))
        {
            // empurra pra longe do inimigo
            float lado = Mathf.Sign(transform.position.x - colisao.transform.position.x);
            TomarDano(1, lado * empurrao);
        }
    }

    public void TomarDano(int dano, float empurraoX)
    {
        if (invencivel > 0f) return;
        vida -= dano;
        invencivel = tempoInvencivel;
        rb.linearVelocity = new Vector2(empurraoX, 5f);
        if (vida <= 0) Morrer();
    }

    public void Morrer()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().name); // recomeça a fase
    }
}
```

- **Pegadinha:** o `Movimento.cs` crava `linearVelocity.x` todo quadro e **apaga o knockback**. O `Movimento` precisa saber que está machucado: um `public bool machucado` lido por ele (`if (machucado)`)

```
return; no começo do Update ) ou o VidaPlayer desligar o Movimento por um instante  
( movimento.enabled = false; ... = true; ).
```

5.10 Roadmap da Fase 2 (combate em tempo real, decidido 03/09)

* **Simples assim**

A ordem em que as peças do combate entram: primeiro o ouvido, depois o molde, o carimbo, o toque fantasma e o inimigo com vida. Cada passo funciona sozinho antes do próximo entrar — e é assim que se testa.

1. ouvido (`Ataque.cs` no player, 5.1) → **2.** molde (prefab, editor, 1.9) → **3.** carimbo (`Instantiate` + `direcao`, 5.2 e 5.3) → **4.** toque fantasma (`OnTriggerEnter2D` no `Espada.cs`, 3.4 e 5.6) → **5.** inimigo com vida (`TomarDano`, 5.7).

Sugestão de Fase 3 (pra quando a 2 fechar): vida na tela (7.3) → som do golpe (7.4) → animação de ataque (7.6) → inimigo que persegue (1.8, `MoveTowards`) → chefe com estados (6.4).

PARTE 6 — TEMPO, CORROTINAS E ESTADOS

6.1 Timers com float e deltaTime — o padrão que já uso [peça em uso]

* Simples assim

Timer é um número que desce todo quadro (`timer -= Time.deltaTime`). Quando chega em zero, pode agir de novo; aí recarrego o número. É o cooldown do golpe, o tempo do dash, o perdão do pulo — tudo a mesma receita de três linhas.

- **O que é:** um número que desce a cada quadro (`x -= Time.deltaTime`) e um `if (x <= 0f)` . É o timer mais simples que existe e resolve 80% dos casos: cooldown, invencibilidade, duração do dash, coyote time.
- **A receita em 3 linhas:**

```
private float timer; // gaveta no topo

void Update()
{
    timer -= Time.deltaTime; // 1. desce sempre
    if (Input.GetButtonDown("Fire1") && timer <= 0f) // 2. só age se zerou
    {
        Atacar();
        timer = 0.35f; // 3. recarrega
    }
}
```

- **Variações:** timer que SOBE (`tempoNoAr += Time.deltaTime`) pra medir quanto tempo algo dura; timer que dispara a cada N s (spawn de inimigo):

```
private float proximoSpawn = 2f;
void Update()
{
    proximoSpawn -= Time.deltaTime;
    if (proximoSpawn <= 0f)
    {
        Instantiate(inimigoPrefab, transform.position, Quaternion.identity);
        proximoSpawn = 2f;
    }
}
```

- **Pegadinhas:** timer que nunca é recarregado fica negativo pra sempre (não é bug, só feio — `Mathf.Max(0f, timer)` se quiser mostrar na tela); dois timers no mesmo nome = um apaga o outro; `Time.timeScale = 0` congela todos.

6.2 Invoke — agendar um método pelo nome [peça futura]

* Simples assim

`Invoke("Nome", 2f)` é "daqui a 2 segundos, chama o método Nome". Sem timer, sem `Update`. Como o nome vai entre aspas, escrever errado = fica mudo — por isso `nameof(Nome)` é melhor: o compilador confere a grafia.

- **O que é:** `Invoke("NomeDoMetodo", segundos)` chama um método meu (sem parâmetros) daqui a N segundos. `InvokeRepeating("Nome", atraso, intervalo)` chama repetidamente. `CancelInvoke()` cancela tudo; `CancelInvoke("Nome")` só aquele.
- **Pra que serve:** "daqui a 2 s, reinicia a fase"; "a cada 1,5 s, nasce um inimigo" — sem timer no `Update`.

```
void Start()
{
    InvokeRepeating("NascerInimigo", 1f, 1.5f); // começa em 1 s, repete a cada 1,5 s
}

void NascerInimigo()
{
    Instantiate(inimigoPrefab, transform.position, Quaternion.identity);
}

void Morrer()
{
    Invoke("Reiniciar", 2f); // dá 2 s de animação antes
}
```

- **Regras / pegadinhas:**
- O nome vai em **string** → senha exata, erro de STRING: `Invoke("nascerInimigo", 1f)` fica mudo (na verdade avisa no Console: "Trying to Invoke method: X couldn't be called"). Preferir `Invoke(nameof(NascerInimigo), 1f)` — o `nameof` faz o compilador conferir.
- Método com parâmetro não pode ser `Invoke` ado. Pra isso, corrotina (6.3).
- `Destroy` no objeto cancela os `Invokes` dele.

6.3 Corrotinas — IEnumerator e yield [peça futura]

* Simples assim

Corrotina é um método que sabe esperar: "fica vermelho, espera 0,1 s, volta ao normal". O `yield return new WaitForSeconds(0.1f)` é a pausa; a Unity continua da linha seguinte quando o tempo passa. Precisa de `StartCoroutine(...)` pra começar — chamar o método sozinho não faz nada.

- **O que é:** um método que **pausa no meio** e continua depois. `yield return new WaitForSeconds(1f) =` "para aqui, volta daqui a 1 s, na linha seguinte". Precisa de `IEnumerator` como tipo de retorno, de `using System.Collections;` no topo, e de ser iniciada com `StartCoroutine(...)`.
- **Pra que serve:** sequências no tempo que ficariam feias com timers: piscar vermelho e voltar, dash (liga → espera → desliga), "3, 2, 1, vai!", spawn em ondas, esperar animação acabar.
- **Os yield que existem:**

linha	espera
<code>yield return null;</code>	o próximo quadro (dentro de um <code>while</code> vira um "Update particular")
<code>yield return new WaitForSeconds(0.5f);</code>	0,5 s de jogo (respeita <code>timeScale</code>)
<code>yield return new WaitForSecondsRealtime(0.5f);</code>	0,5 s reais (ignora pausa)
<code>yield return new WaitForFixedUpdate();</code>	o próximo passo da física
<code>yield return new WaitUntil(() => estaNoChao);</code>	até a condição virar verdadeira
<code>yield break;</code>	encerra a corrotina agora

```

using System.Collections;                                     // ← sem isso, IEnumerator não existe (CS0246)
using UnityEngine;

public class Exemplo : MonoBehaviour
{
    public SpriteRenderer sprite;

    // 1) Piscar ao tomar dano
    public void TomarDano()
    {
        StartCoroutine(Piscar());                             // INICIA – chamar Piscar() sozinho não faz nada
    }
    IEnumerator Piscar()
    {
        sprite.color = Color.red;
        yield return new WaitForSeconds(0.1f);
        sprite.color = Color.white;
    }

    // 2) Dash com corrotina (a versão limpa do 4.5)
    IEnumerator Dash(Rigidbody2D rb, float direcao)
    {
        float gravidade = rb.gravityScale;
        rb.gravityScale = 0f;
        rb.linearVelocity = new Vector2(direcao * 25f, 0f);
        yield return new WaitForSeconds(0.15f);
        rb.gravityScale = gravidade;
    }

    // 3) Contagem regressiva
    IEnumerator Contagem()
    {
        for (int i = 3; i > 0; i--)
        {
            Debug.Log(i);
            yield return new WaitForSeconds(1f);
        }
        Debug.Log("vai!");
    }

    // 4) "Update particular": segue o alvo por 2 s e para
    IEnumerator Seguir(Transform alvo)
    {
        float fim = Time.time + 2f;
        while (Time.time < fim)
        {
            transform.position = Vector2.MoveTowards(transform.position, alvo.position, 3f * Time.deltaTime);
            yield return null;                                 // sem isso o while congela a Unity
        }
    }
}

```

```

    }
}
}

```

• Regras / pegadinhas:

- `Piscar();` **sozinho não roda nada** — só cria o "roteiro". Tem que ser `StartCoroutine(Piscar());`.
- `while` dentro de corrotina **precisa** de um `yield return null` dentro — senão é o while infinito que congela o editor (1.5).
- Parar: guardar `Coroutine c = StartCoroutine(X()); StopCoroutine(c);` ou `StopAllCoroutines()`. Chamar `StartCoroutine(Piscar())` de novo antes de acabar cria uma SEGUNDA rodando junto — pra "reiniciar", parar a antiga primeiro.
- Corrotina morre junto com o objeto (`Destroy`) e com `SetActive(false)` — não volta no `SetActive(true)`.
- Corrotina roda no objeto **onde foi iniciada**; `Destroy(gameObject)` na primeira linha de uma corrotina mata o resto dela.
- Ela pode ter parâmetros (ao contrário do `Invoke`).

6.4 enum + switch — a máquina de estados simples [peça futura]

* Simples assim

`enum` é uma lista de opções com nome (Parado, Andando, Pulando). Uma variável desse tipo só aceita uma opção da lista — o boneco está em UM estado por vez. O `switch` olha o estado e roda só o bloco daquele caso. Substitui seis `bool` que se contradizem.

- **O que é:** `enum` é uma lista de nomes com valor fixo: `enum Estado { Parado, Andando, Pulando, Dash, Atacando, Machucado }`. Uma caixa desse tipo só aceita um desses nomes — o compilador confere (`Estado.Correndo` não existe → erro barulhento, não `string`).
- **Pra que serve:** parar de ter 6 `bool` que se contradizem (`dando && pequeno && machucado`) e ter UMA resposta: "em que estado o boneco está?". Cada estado decide o que pode acontecer.

```

public enum Estado { Parado, Andando, Pulando, Dash, Machucado }

public class Boneco : MonoBehaviour
{
    public Estado estado = Estado.Parado;    // enum público vira dropdown no Inspector

    void Update()
    {
        switch (estado)
        {
            case Estado.Parado:
            case Estado.Andando:              // os dois caem no mesmo bloco
                Andar();
                if (Input.GetButtonDown("Jump") && estaNoChao) estado = Estado.Pulando;
                if (Input.GetButtonDown("Dash")) estado = Estado.Dash;
                break;

            case Estado.Pulando:
                Andar();                      // controle no ar
                if (estaNoChao) estado = Estado.Parado;
                break;

            case Estado.Dash:
                Dashando();                   // quando acabar, ele mesmo volta pra Parado
                break;
        }
    }
}

```

```

        case Estado.Machucado:
            // nada: espera o timer devolver pra Parado
            break;
    }
}

```

- **Regras / pegadinhas:** todo `case` termina com `break` (CS0163 se esquecer); `enum` começa com maiúscula e mora fora ou dentro da classe (fora = todo script enxerga); comparar com `==` (`if (estado == Estado.Dash)`); `Debug.Log(estados)` imprime o nome, ótimo pra depurar.

6.5 static e o GameManager — uma caixa só pro jogo inteiro [peça futura]

* Simples assim

`static` é uma caixa que existe uma vez só pro jogo inteiro, pendurada na CLASSE e não num objeto. Todo script alcança `GameManager.pontos` sem arrastar nada. Bom pra pontos, vidas, pausa. Cuidado: ela sobrevive à troca de cena.

- **O que é:** `static` = a caixa pertence à CLASSE, não a cada objeto. `public static int pontos;` é uma caixa única que todo script lê como `GameManager.pontos` (coleira com o nome da classe, sem caixinha, sem arrastar).
- **Pra que serve:** pontos, vidas, "está pausado?" — coisas que só existem uma vez e que a moeda, a UI e o inimigo precisam alcançar sem se conhecerem.
- **O padrão "Instance" (singleton):** um objeto `GameManager` na cena com um script que guarda a si mesmo numa caixa `static` — daí todo mundo chama `GameManager.Instance.AdicionarPontos(1)`.

```

using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;           // a caixa única que aponta pro objeto da cena

    public int pontos;
    public int vidas = 3;

    void Awake()
    {
        Instance = this;                          // "eu sou o GameManager"
    }

    public void AdicionarPontos(int quanto)
    {
        pontos += quanto;
        Debug.Log("pontos: " + pontos);
    }
}

// na Moeda:
// GameManager.Instance.AdicionarPontos(1);

```

- **Regras / pegadinhas:**
- `static` NÃO aparece no Inspector e não é caixinha — é por isso que não precisa arrastar.
- Caixa `static` **sobrevive à troca de cena** (a classe não morre) — pontos que "vazam" pra fase seguinte é isso. Zerar no `Start` da fase, se for o caso.

- Método static não enxerga `transform`, `gameObject` nem campos normais (CS0120 "referência de objeto necessária"). Só o padrão `Instance` resolve isso.
- Dois `GameManager` na cena = o segundo `Awake` sobrescreve `Instance`. Um só.

6.6 Listas e arrays — várias caixas do mesmo tipo [peça futura]

* Simples assim

Array (`Transform[]`) é uma prateleira com número fixo de vagas; `List<T>` é uma prateleira que cresce e encolhe (`Add`, `Remove`). As vagas começam em 0: numa prateleira de 3, as vagas são 0, 1 e 2 — pedir a vaga 3 dá erro.

- **Array** `T[]` : tamanho fixo. `public GameObject[] pontosDeSpawn;` vira caixinha com tamanho ajustável no Inspector, e eu arrasto os objetos.
- `List<T>` : tamanho vivo (`Add`, `Remove`, `Count`). Precisa de `using System.Collections.Generic;` `public List<GameObject> inimigos;` também vira caixinha.

```
using System.Collections.Generic;
using UnityEngine;

public class Spawner : MonoBehaviour
{
    public Transform[] pontos;           // array: arrasto 3 objetos vazios no Inspector
    public GameObject inimigoPrefab;
    private List<GameObject> vivos = new List<GameObject>(); // lista: nasce vazia

    void NascerUm()
    {
        Transform ponto = pontos[Random.Range(0, pontos.Length)]; // índice de 0 a Length-1
        GameObject novo = Instantiate(inimigoPrefab, ponto.position, Quaternion.identity);
        vivos.Add(novo);
    }

    void MatarTodos()
    {
        foreach (GameObject inimigo in vivos) Destroy(inimigo);
        vivos.Clear();
    }

    int QuantosVivos()
    {
        vivos.RemoveAll(i => i == null); // limpa os que já morreram (falso null)
        return vivos.Count;
    }
}
```

• Regras / pegadinhas:

- Índice começa em **0**: `pontos[0]` é o primeiro, `pontos[pontos.Length - 1]` o último; `pontos[3]` num array de 3 → `IndexOutOfRangeException`.
- Array usa `.Length`, lista usa `.Count`. Trocar um pelo outro dá CS1061.
- Remover item de uma lista **dentro de um** `foreach` **sobre ela** → `InvalidOperationException` ("collection was modified"). Usar `RemoveAll` ou um `for` de trás pra frente.
- `new List<GameObject>()` no topo evita `NullReferenceException` no primeiro `Add` — lista `public` a Unity cria sozinha; `private` sou eu que crio.

PARTE 7 — PEÇAS DE APOIO: CÂMERA, CENAS, UI, SOM, ANIMAÇÃO

7.1 Câmera que segue o boneco [peça futura]

* Simples assim

A câmera segue o boneco copiando a posição dele todo quadro — com suavidade (Lerp) e sem copiar o z, senão fica em cima dele e a tela apaga. Roda no `LateUpdate` porque nesse ponto o boneco já andou no quadro.

```
using UnityEngine;

public class Camera2D : MonoBehaviour // pendurar na Main Camera
{
    public Transform alvo; // arrastar o Boneco
    public float suavidade = 8f;
    public Vector2 deslocamento = new Vector2(0f, 1f);

    void LateUpdate() // DEPOIS do boneco andar no quadro
    {
        Vector3 destino = new Vector3(alvo.position.x + deslocamento.x,
                                      alvo.position.y + deslocamento.y,
                                      transform.position.z); // MANTÉM o z da câmera (-10)
        transform.position = Vector3.Lerp(transform.position, destino, suavidade * Time.deltaTime);
    }
}
```

- **Pegadinhas:** copiar o z do boneco (0) põe a câmera em cima dele e a tela fica preta — sempre manter o `transform.position.z` da câmera; câmera no `Update` treme (o boneco ainda não andou); `Rigidbody` do player com `Interpolate` tira o tremor que sobra. O pacote **Cinemachine** faz isso melhor (limites de fase, zona morta) — peça pra depois.

7.2 Cenas — SceneManager [peça futura]

* Simples assim

Cada fase ou menu é uma cena. `SceneManager.LoadScene("Fase2")` fecha a atual e abre a outra. A cena precisa estar cadastrada na lista de build, e trocar de cena destrói tudo da anterior (menos o que é `static` ou marcado com `DontDestroyOnLoad`).

- **O que é:** cada fase/menu é uma **cena** (`.unity`). `SceneManager.LoadScene("Fase2")` troca a atual pela nova. Precisa de `using UnityEngine.SceneManagement;`
- **Registrar a cena (sem isso o `LoadScene` grita):** Unity 6: **File** → **Build Profiles** → aba *Scene List* → *Add Open Scenes* (versões antigas: *File* → *Build Settings* → *Scenes In Build*). Cada cena ganha um número (índice) na ordem da lista.

```
using UnityEngine.SceneManagement;

SceneManager.LoadScene("Fase2"); // pelo nome (senha exata)
SceneManager.LoadScene(1);      // pelo índice da lista
```

```
SceneManager.LoadScene(SceneManager.GetActiveScene().name); // recarregar a atual (morreu)
SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1); // a próxima da lista
```

- **Pegadinhas:** nome errado ou cena fora da lista → "Scene 'X' couldn't be loaded"; trocar de cena **destrói tudo** da cena velha (exceto `static` e objetos com `DontDestroyOnLoad(gameObject)`); o `GameManager` que precisa sobreviver usa `DontDestroyOnLoad` no `Awake`.

7.3 UI rápida — texto na tela com TextMeshPro [peça futura]

* Simples assim

Texto na tela é um objeto de UI dentro de um Canvas. No script, uma caixinha `TextMeshProUGUI` e `texto.text = "Vida: 3"`. O erro clássico é a caixinha ser de um tipo (`Text`) e o objeto ser de outro (`TextMeshProUGUI`).

1. Hierarchy → botão direito → **UI → Text - TextMeshPro** (a Unity cria o `Canvas` e o `EventSystem` sozinha; se pedir *Import TMP Essentials*, aceitar).
2. Posicionar pelo **Rect Transform** (as âncoras no canto do Inspector: clicar no quadrado das âncoras e escolher canto superior esquerdo pra HUD).
3. No script: `using TMPro;` e uma caixinha `public TextMeshProUGUI textoVida;` — arrastar o objeto de texto.

```
using TMPro;
using UnityEngine;

public class HUD : MonoBehaviour
{
    public TextMeshProUGUI textoVida;
    public TextMeshProUGUI textoPontos;

    public void Atualizar(int vida, int pontos)
    {
        textoVida.text = "Vida: " + vida;
        textoPontos.text = $"Pontos: {pontos}";
    }
}
```

- **Pegadinhas:** `Text` (legado) e `TextMeshProUGUI` são tipos diferentes — a caixinha tem que ser do tipo do componente que está no objeto (o novo é sempre TMP); o texto do Canvas não aparece na Scene do jogo, só na aba Game (o Canvas é gigante na Scene — normal); `Canvas Scaler` em *Scale With Screen Size* pra não ficar minúsculo em tela grande.

7.4 Som — AudioSource [peça futura]

* Simples assim

Um `AudioSource` no boneco é a caixa de som; os arquivos `.wav` viram `AudioClip`.

`som.PlayOneShot(somPulo)` toca um efeito por cima do que já toca. Só pode existir um `AudioListener` (o "ouvido") na cena — ele já vem na câmera.

- Um `AudioSource` no Boneco (Add Component), `Play On Awake` desmarcado. Sons (`.wav` / `.ogg`) na pasta Assets viram `AudioClip`.

```
public AudioSource som;           // arrastar o Boneco (a Unity acha o AudioSource)
public AudioClip somPulo;
public AudioClip somGolpe;

void Pular()
{
    som.PlayOneShot(somPulo);    // toca por cima, sem cortar o anterior
}
```

- **Pegadinhas:** só UM `AudioListener` por cena (já vem na Main Camera — outra câmera com listener gera aviso); `som.Play()` toca o clip da caixinha `AudioClip` do componente e corta o que estava tocando; `PlayOneShot` é o certo pra efeitos; música de fundo = outro `AudioSource` com `Loop`.

7.5 SpriteRenderer — o desenho [peça de apoio]

* Simples assim

`SpriteRenderer` é o componente que desenha. Por ele eu troco a imagem, tinjo de vermelho, espelho e decido quem fica na frente (*Order in Layer*). Esconder o desenho (`enabled = false`) não desliga o objeto — a física continua.

membro	faz
<code>sprite.flipX = true</code>	espelha só o desenho (4.7)
<code>sprite.color = Color.red</code>	tinge (pisca de dano) — <code>new Color(1f, 1f, 1f, 0.5f)</code> = meio transparente
<code>sprite.sprite = outroSprite</code>	troca a imagem (agachado, machucado)
<code>sprite.sortingOrder = 5</code>	ordem de desenho: maior = na frente

membro	faz
<code>sprite.enabled = false</code>	esconde sem desativar o objeto (pisca de invencível)

- **Pegadinha:** *Sorting Layer* e *Order in Layer* decidem quem fica na frente — não o z. Fundo em ordem -10, chão 0, boneco 5, golpe 6.

7.6 Animator — parâmetros e transições [peça em uso, aula de 04/09: IsJump e IsFall]

(04/09/2026 — sistema de animação centralizado no `Update do Movimento.cs`)

* Simples assim

O Animator é o DJ das animações: ele tem os "discos" (Idle, Jump, Fall) e as setas entre eles. O script não escolhe o disco — só mexe nos **interruptores** (`IsJump` , `IsFall`) e o Animator troca sozinho quando as condições das setas batem. Minha parte é decidir, todo quadro: estou subindo no ar? caindo? no chão? E ligar o interruptor certo.

- **O que é:** o `Animator` é uma máquina de estados de ANIMAÇÕES (Idle, Run, Jump, Fall, Attack). O script não escolhe a animação: ele **mexe nos parâmetros** (`bool` , `float` , `trigger`) e as transições (setas na janela Animator) reagem.

- **Onde:** Window → Animation → **Animator**. Cada estado é um clip; cada seta tem *Conditions* com os parâmetros.
- **A lógica de estados (a decisão fica no script, num lugar só):** todo quadro, no `Update`, olho a velocidade vertical (`rb.linearVelocity.y`) e o chão (`estaNoChao`) e ligo/desligo `IsJump` e `IsFall` :

situação	como sei	IsJump	IsFall
subindo no ar	<code>!estaNoChao && rb.linearVelocity.y > 0</code>	true	false
caindo no ar	<code>!estaNoChao && rb.linearVelocity.y < 0</code>	false	true
no chão (<code>else</code>)	qualquer outro caso	false	false

```
[Header("Animação")]
public Animator animator;           // arrastar o Boneco (a Unity acha o componente Animator)

void Update()
{
    estaNoChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao); // 3.5
    // ... andar, virar, pular ...
    Animar();                       // por último: decide com a velocidade já deste quadro
}

void Animar()
{
    if (!estaNoChao && rb.linearVelocity.y > 0.1f)           // subindo no ar
    {
        animator.SetBool("IsJump", true);
        animator.SetBool("IsFall", false);
    }
    else if (!estaNoChao && rb.linearVelocity.y < -0.1f)      // caindo no ar
    {
        animator.SetBool("IsJump", false);
        animator.SetBool("IsFall", true);
    }
    else                                                       // no chão
    {
        animator.SetBool("IsJump", false);
        animator.SetBool("IsFall", false);
    }
}
```

Montar a máquina na janela Animator

1. **Controller:** Project → botão direito → Create → Animation → Animator Controller → BonecoAnimator. No Boneco: Add Component → Animator → arrastar o controller na caixa Controller.
2. **Clips:** arrastar as animações (Idle, Jump, Fall) pra dentro da janela Animator — cada uma vira um estado (caixa). Botão direito no Idle → Set as Layer Default State (fica laranja = começa nele).
3. **Parâmetros:** aba Parameters (canto esquerdo) → + → Bool → IsJump; + → Bool → IsFall. A grafia tem que ser IGUAL à do código (I, J e F maiúsculos).
4. **Setas:** botão direito no estado → Make Transition → clicar no destino. Em cada seta, no Inspector: Conditions → + → escolher o parâmetro e true / false:
 - Idle → Jump: IsJump = true
 - Jump → Fall: IsFall = true
 - Fall → Idle: IsFall = false
 - Idle → Fall: IsFall = true (cair da beirada sem pular)
 - Jump → Idle: IsJump = false e IsFall = false (pulo curtinho que pousa sem "cair")
5. **Em toda seta:** desmarcar **Has Exit Time** e deixar *Transition Duration* = 0 — senão a troca espera o clip acabar e a animação "atrasa".
6. **Testar:** Play com a janela Animator aberta — a barrinha azul mostra o estado atual e os parâmetros mudam ao vivo. Pulou e não trocou? Conferir o nome do parâmetro letra por letra.

Mais fundo: os outros parâmetros (peça futura)

```
animator.SetBool("andando", Mathf.Abs(rb.linearVelocity.x) > 0.01f); // Idle ↔ Run
animator.SetFloat("velY", rb.linearVelocity.y); // blend tree de pulo/queda
animator.SetTrigger("ataque"); // dispara uma vez e reseta
sozinho
```

• Regras / pegadinhas:

- Nome do parâmetro é **senha exata** em string: SetBool("isJump", ...) com i minúsculo, se o parâmetro se chama IsJump, é **silencioso** (a Unity avisa "Parameter 'isJump' does not exist" no Console em alguns casos; a animação fica parada sempre).
- **Por que > 0.1f e não > 0:** parado no chão a velocidade y oscila em números minúsculos (0.0001, -0.0001) e o IsFall piscaria. A folga de 0.1 mata o piscar.
- **Por que checar !estaNoChao também:** descendo uma rampa a velocidade y é negativa mesmo com os pés no chão — sem o !estaNoChao, o boneco "cai" andando. No primeiro quadro do pulo o círculo ainda toca o chão, então o IsJump pode ligar um quadro depois — imperceptível.
- Chamar Animar() **depois** de andar/pular no Update: a velocidade já é a deste quadro. Chamado antes, a animação fica um quadro atrasada.
- SetBool todo quadro é normal e barato — o Animator só troca de estado quando a condição da seta muda.
- Nas setas, desmarcar **Has Exit Time** pra transição responder na hora (senão espera o clip acabar); *Transition Duration* 0 em pixel art.
- SetTrigger numa transição sem seta de volta trava no estado. Toda animação de ataque precisa da seta de volta pro Idle (com *Has Exit Time* ligado nessa).
- Animação que mexe em transform.position briga com o Rigidbody — animar só o sprite.
- Caixa animator vazia = NullReferenceException na primeira SetBool. Alternativa: animator = GetComponent<Animator>(); no Awake (5.4).

PARTE 8 — DICIONÁRIO DE ERROS

8.1 Erros de compilação (gritam vermelho no Console, o jogo nem dá Play)

* Simples assim

Erro vermelho no Console antes do Play = o C# não entendeu alguma linha e o jogo nem começa. A mensagem tem um código (CS1002) e a linha. Duplo-clique leva até ela; muitas vezes o erro de verdade está na linha de cima (o ; esquecido).

Ler a mensagem de trás pra frente: **arquivo, linha, código CS, texto**. Duplo-clique leva à linha. O erro costuma estar **na linha apontada ou na anterior** (ponto-e-vírgula esquecido aparece na linha de baixo).

código	mensagem (resumo)	causa comum	conserto
CS0103	"o nome 'x' não existe no contexto atual"	esqueci a coleira (<code>linearVelocity</code> solto), errei a letra, variável declarada dentro de outro método	<code>rb.linearVelocity</code> ; conferir nome; subir a variável pra classe
CS1955	"membro não invocável 'x' não pode ser usado como método"	chamei uma CAIXA com parênteses: <code>transform.localPosition(...)</code>	<code>transform.localPosition = new Vector3(...)</code>
CS1002	"; esperado"	faltou o ponto-e-vírgula na linha ANTERIOR	pôr ;
CS1513	"} esperado"	abri chave e não fechei	contar chaves; Shift+Alt+F formata e mostra o buraco
CS1525	"termo de expressão inválido"	sintaxe quebrada: <code>if x > 0, new Vector2(3f,)</code>	parênteses no if, argumento faltando
CS0029	"não é possível converter 'double' em 'float'"	número sem f: <code>velocidade = 5.5;</code>	<code>5.5f</code>
CS0266	"conversão implícita de 'float' em 'int'... existe uma conversão explícita"	<code>int vida = 2.5f;</code>	<code>(int)</code> ou <code>Mathf.RoundToInt</code>
CS0246	"tipo ou namespace 'x' não encontrado"	faltou using: <code>IEnumerator</code> sem <code>System.Collections</code> , <code>List</code> sem <code>System.Collections.Generic</code> , <code>TextMeshProUGUI</code> sem <code>TMPPro</code> , <code>SceneManager</code> sem <code>UnityEngine.SceneManagement</code>	adicionar o using no topo
CS0120	"referência de objeto necessária para membro não estático"	usei <code>transform</code> /campo normal dentro de método <code>static</code> , ou <code>Movimento.direcao</code> (classe) em vez de <code>movimento.direcao</code> (objeto)	tirar o <code>static</code> , ou usar a caixinha do objeto
CS0122	"'x' é inacessível devido ao nível de proteção"	vizinho lendo campo/método <code>private</code>	virar <code>public</code>

código	mensagem (resumo)	causa comum	conserto
CS0165	"uso de variável local não atribuída"	declarei <code>float x;</code> e li antes de dar valor	<code>float x = 0f;</code>
CS0161	"nem todos os caminhos de código retornam um valor"	método <code>bool / float</code> sem <code>return</code> em algum <code>if</code>	<code>return</code> no fim
CS0111	"já define um membro chamado 'Update'"	dois <code>Update</code> (ou dois métodos iguais)	apagar um
CS0116	"um namespace não pode conter diretamente membros"	código fora das chaves da classe	mover pra dentro
CS0128	"variável local 'x' já definida neste escopo"	declarei <code>float lado</code> duas vezes no mesmo método	tirar o tipo da segunda (<code>lado = ...</code>)
CS0201	"somente atribuição, chamada... podem ser usadas como instrução"	linha que não faz nada: <code>vida + 1;</code> ou <code>Pular;</code>	<code>vida += 1;</code> / <code>Pular();</code>
CS1061	"'x' não contém definição para 'y'"	membro que não existe: <code>rb.velocidade</code> , <code>lista.Length</code> , <code>array.Count</code>	<code>rb.linearVelocity</code> , <code>.Count</code> em lista, <code>.Length</code> em array
CS0117	"'KeyCode' não contém definição para 'Espaco'"	nome errado num enum	<code>KeyCode.Space</code> (Ctrl+Espaço no VS Code lista)
CS1503	"não é possível converter argumento de 'x' em 'y'"	passsei o tipo errado: <code>Instantiate("golpe", ...)</code> , <code>TomarDano(1.5f)</code>	passar o tipo pedido
CS1501	"nenhuma sobrecarga para 'x' aceita N argumentos"	chamei com quantidade errada de parâmetros	conferir a assinatura
CS0163	"controle não pode passar de um rótulo de case para outro"	<code>case</code> sem <code>break</code>	pôr <code>break;</code>
CS1612	"não é possível modificar o valor de retorno de 'x' porque não é uma variável"	<code>rb.linearVelocity.y = 5f;</code>	carimbar o vetor inteiro (as 3 irmãs)
CS0428	"não é possível converter grupo de métodos"	esqueci os parênteses: <code>if (EstaVivo)</code>	<code>EstaVivo()</code>

código	mensagem (resumo)	causa comum	conserto
CS0106	"o modificador 'public' não é válido para este item"	<code>public bool x</code> declarado DENTRO de um método (a faxina de 04/09)	tirar o <code>public</code> — e o tipo também, se a caixa já existe no topo (1.11)
CS1519	"token inválido '=' na declaração de membro"	instrução solta fora de método: <code>estaNoChao = true;</code> no topo da classe	mover a linha pra dentro de um método (1.11)

8.2 Erros em tempo de execução (só aparecem com o jogo RODANDO)

* Simples assim

Esses só aparecem com o jogo rodando: o código compilou, mas na hora H uma caixinha estava vazia, uma tag não existia, um objeto já tinha morrido. O Console aponta a linha; o conserto quase sempre é no Inspector (arrastar, criar a tag) ou um `if` de guarda.

mensagem	causa	conserto
NullReferenceException: Object reference not set...	caixinha vazia (<code>rb</code> , <code>golpePrefab</code> , <code>movimento</code> , <code>pe</code>), <code>GetComponent</code> que não achou, <code>FindWithTag</code> que não achou	Inspector → arrastar; guardar com <code>if (x != null)</code>

mensagem	causa	conserto
MissingReferenceException	usei um objeto que já foi <code>Destroy</code> ado	checar <code>== null</code> antes; não guardar referência pra coisa que morre
MissingComponentException: There is no 'Rigidbody2D' attached	<code>GetComponent</code> no objeto sem o componente	Add Component
ArgumentException: Input Button X is not setup	apelido de botão errado/ inexistente	tabela 2.2, criar em 2.6
ArgumentException: Input Axis X is not setup	idem pra régua	idem
UnityException: Tag: X is not defined	<code>CompareTag</code> com tag que não existe na lista	criar a tag (3.3)
InvalidOperationException: You are trying to read Input using the UnityEngine.Input class...	projeto no novo Input System	2.7
Scene 'X' couldn't be loaded	cena fora da lista de build ou nome errado	7.2
IndexOutOfRangeException	<code>array[3]</code> num array de 3	índices vão de 0 a <code>Length-1</code>
InvalidOperationException: Collection was modified	removi item de lista dentro de <code>foreach</code>	<code>RemoveAll</code> / <code>for</code> de trás pra frente
Coroutine couldn't be started because the game object 'X' is inactive	<code>StartCoroutine</code> em objeto desativado	ativar antes, ou iniciar de outro objeto

mensagem	causa	conserto
Trying to Invoke method: X couldn't be called	nome errado no <code>Invoke</code>	<code>nameof(Metodo)</code>
Can't add script component 'X' because the script class cannot be found	nome do arquivo $\neq$ nome da classe, ou erro de compilação no projeto	igualar nomes; corrigir os vermelhos primeiro
The referenced script (Unknown) on this Behaviour is missing!	script apagado/renomeado por fora da Unity, ou <code>.meta</code> perdido	reatribuir o script no componente; nunca apagar <code>.meta</code>
SendMessage cannot be called during Awake/OnEnable/OnValidate (aviso)	mexi em componente de outro objeto no <code>Awake</code>	mover pro <code>Start</code>

8.3 Bugs silenciosos (sem mensagem nenhuma — o mais perigoso)

* Simples assim

Sem erro nenhum e nada acontece — o pior tipo. Quase sempre é um nome escrito diferente do esperado (`update`, `"jump"`, `IsJump` $\times$ `isJump`), uma caixinha errada no Inspector ou uma variável escrita em dois lugares. Esta tabela lista os sintomas; o remédio é `Debug.Log` até achar onde o código para de passar.

sintoma	causa provável	onde olhar
o alarme nunca toca	nome de evento errado (<code>update</code> , <code>OnCollisionEnter</code> sem 2D, <code>OnTriggerEnter2D(Collision2D)</code> com o tipo errado)	1.1, 3.2, 3.4 — comparar letra por letra; <code>Debug.Log</code> na 1ª linha
tecla não faz nada	<code>GetKeyDown</code> no <code>FixedUpdate</code> ; apelido errado que não é string de erro; script desligado; objeto desativado	1.1, 2.1
mudei o valor no código e nada muda	o Inspector vence	1.3 — mudar lá ou Reset
ajustei no Inspector rodando e sumiu	Play Mode não salva	1.3 — Copy/Paste Component Values
<code>if</code> do chão nunca entra	tag criada mas não colada; <code>"ground"</code> minúsculo	3.3
<code>estaNoChao</code> nunca liga (círculo)	LayerMask vazia; chão fora da layer; <code>checadorDeChao</code> na posição errada ou caixinha vazia	3.5 — olhar o gizmo amarelo
<code>estaNoChao</code> sempre ligado	player na mesma layer do chão; raio grande demais	3.5
pulo infinito	<code>estaNoChao</code> ligada por trombada (parede conta como chão) ou raio do círculo grande demais	3.5
pulo no ar ao cair da beirada	o furo antigo: ground check por trombada sem <code>Exit</code> . Fechado em 04/09 pelo círculo — se voltou, o <code>OnCollisionEnter2D</code> voltou a escrever em <code>estaNoChao</code>	3.5, 1.11

sintoma	causa provável	onde olhar
boneco pula grudado na parede / "sobe" a parede	OnCollisionEnter2D contando a lateral como chão	3.5
estaNoChao pisca (liga e desliga sozinha)	dois donos: o Update calcula E o alarme escreve	1.11
mudo a variável dentro do alarme e o Update não vê	redeclarei com bool dentro do método (sombra)	1.11
agachar faz o boneco flutuar / afundar	pivô do sprite no Center; collider sem offset	4.6
animação de pulo não vira queda (ou pisca parado)	condição da seta errada; Has Exit Time ligado; sem a folga de 0.1 na velocidade; nome IsJump / IsFall diferente	7.6
boneco flutua na queda	corte do pulo sem o guarda > 0	4.1
boneco flutua pra sempre depois do dash	gravityScale não devolvido	4.5
boneco tomba/rola	Freeze Rotation Z desligado	1.7
boneco gruda na parede no ar	atrito — Physics Material SemAtrito (Friction 0) no collider do boneco	3.1
boneco atravessa parede	teleporte (localPosition / Translate) ou velocidade alta com Discrete	1.8, 1.7
boneco desliza ao soltar a tecla	GetAxis em vez de.GetAxisRaw	2.3
knockback não acontece	o andar crava linearVelocity.x no quadro seguinte	1.7, 5.9
golpe nasce sempre pra direita	+ fixo sem * direcao	1.8, 5.3
golpe cai pelo chão	Rigidbody2D Dynamic no golpe	3.4 — Kinematic
golpe acerta o próprio player	matriz de layers	3.3
golpe acerta duas vezes	sem a trava jaBateu	5.6
Hierarchy enche de (Clone)	sem Destroy	5.5
animação não muda	nome do parâmetro do Animator errado; Has Exit Time	7.6
som não toca	Play On Awake ? AudioListener faltando? volume 0?	7.4
câmera preta / boneco some	z da câmera copiado do boneco; sprite atrás da câmera	7.1, 1.8

sintoma	causa provável	onde olhar
tudo congelou	while infinito	1.5 — Gerenciador de Tarefas, e salvar antes do próximo Play
funciona em casa, não no trabalho	stub sem Input Manager/Console; config do VS Code não viaja	9.2, 9.5

8.4 Checklist: "meu boneco não anda"

* Simples assim

Quando o boneco não anda, não adivinho: percorro a lista de cima pra baixo, uma pergunta por vez. Nove em dez vezes o problema está nas cinco primeiras.

1. O script está **pendurado** no Boneco? (Inspector mostra Movimento (Script) ?)
2. O checkbox do script e o do objeto estão **marcados**?
3. O Console está **limpo de vermelho**? (um erro de compilação em QUALQUER script trava todos)
4. A caixinha `rb` está **preenchida**?
5. `velocidade` no **Inspector** é maior que 0? (o Inspector vence o código)
6. O `Update` está escrito `Update` com U maiúsculo e sem parâmetros?
7. O apelido é `"Horizontal"` com H maiúsculo?
8. O Rigidbody2D é **Dynamic** e não está com *Freeze Position X*?
9. Dei Play? (a aba Game está mostrando o jogo rodando — botão azul?)
10. `Debug.Log(lado)` no `Update` imprime -1/0/1 ao apertar A/D? Se imprime e não anda, o problema é da linha do `linearVelocity` pra baixo; se não imprime, é do Input pra cima.

PARTE 9 — FERRAMENTAS: VS CODE, EDITOR DA UNITY, GIT

9.1 O ponto que aceita sugestão sozinho

(03/09/2026 — o capeta dos usings fantasmas)

* Simples assim

O VS Code aceita a sugestão sozinho quando digito `.` — e cola coisa que ninguém pediu (um `using` fantasma, uma palavra errada). Desligar a opção *Accept Suggestion On Commit Character* faz a sugestão entrar só com Tab/Enter, quando EU quiser.

- **O que acontece:** com a caixinha de sugestões aberta, digitar `.` significa "aceita a sugestão marcada E põe o ponto" — cola bobagem que ninguém pediu. Foi ele que criou o `using System.Numerics` fantasma e o `==` cenas do Python.
- **Piora no trabalho:** o csproj da Unity aponta DLLs que só existem no PC de casa → o VS Code não conhece `Input` de verdade e chuta palavras soltas.
- **Conserto (fazer nos DOIS PCs, a config não viaja no git):** `Ctrl + ,` → pesquisar `commit character` → **desmarcar** "Editor: Accept Suggestion On Commit Character". Aceitar sugestão vira ato voluntário: só com `Tab` / `Enter`.

Mais fundo: outros usings fantasmas pra reconhecer

apareceu no topo	veio de	apaga?
<code>using System.Numerics;</code>	<code>Vector2</code> errado (o do .NET, não o da Unity)	sim — o <code>Vector2</code> certo vem de <code>UnityEngine</code>
<code>using UnityEditor;</code>	sugestão de coisa de editor	sim — quebra o build final
<code>using System.Linq;</code>	<code>.First()</code> , <code>.Where()</code>	só se eu usei de propósito
<code>using UnityEngine.UIElements;</code>	<code>Button</code> / <code>Label</code> errados	sim (UI Toolkit, outro mundo)

Os únicos `using` que os meus scripts precisam hoje: `UnityEngine` sempre; `System.Collections` (corrotinas); `System.Collections.Generic` (listas); `TMPro` (texto); `UnityEngine.SceneManagement` (cenas).

9.2 IntelliSense de verdade (em casa) e o stub (no trabalho)

* Simples assim

IntelliSense é o VS Code conhecer a Unity de verdade (completar `Input.`, mostrar o erro antes de salvar). Em casa, com a Unity instalada, ele funciona depois do *Regenerate project files*; no trabalho, o stub só finge — compila, mas não sabe nada da Unity. Escrever no trabalho, testar em casa.

- **Em casa, pro VS Code conhecer** `Input`, `Rigidbody2D` **e companhia:** 1. Na Unity: Edit → Preferences → **External Tools** → *External Script Editor* = Visual Studio Code; marcar *Embedded packages*, *Registry packages* (opcional) e clicar em **Regenerate project files**. 2. No VS Code: extensão **C# Dev Kit** (Microsoft) + extensão **Unity** (Microsoft). Precisa do **.NET SDK** instalado no PC.

3. Abrir a PASTA do projeto (`aula_8`), não o arquivo solto. Esperar a bolinha de carregamento do C# no rodapé sumir. 4. Sumiu o IntelliSense de novo? Repetir o *Regenerate project files* — é ele que reescreve o `.csproj` com os caminhos das DLLs.

- **No trabalho:** sem a Unity instalada, o `.csproj` aponta pra DLLs que não existem → o C# vira "cego" e chuta sugestões (o veneno do 9.1). O **stub** (a `UnityEngine` de mentira que só compila) serve pra pegar erro de compilação; não serve pra testar Input, Console, física. Regra: no trabalho escrevo e compilo; em casa dou Play.

9.3 Atalhos do VS Code que pagam a conta

* Simples assim

Quatro atalhos pagam a conta: `Ctrl + .` conserta o `using` faltando, `Ctrl + Espaço` mostra as opções, `Shift + Alt + F` arruma a indentação (e revela a chave faltando), `Ctrl + /` comenta a linha.

atalho	faz
<code>Ctrl + .</code>	lâmpada: sugere o <code>using</code> faltando / conserto rápido
<code>Ctrl + Espaço</code>	abre a lista de sugestões (ver os nomes do <code>KeyCode</code> , membros do <code>rb</code>)
<code>F12</code>	vai pra definição (clicar em <code>linearVelocity</code> mostra o que é)
<code>Shift + Alt + F</code>	formata o arquivo: indentação certa mostra chave faltando
<code>Ctrl + /</code>	comenta/descomenta a linha (<code>//</code>)
<code>Alt + ↑ / ↓</code>	move a linha
<code>Ctrl + D</code>	seleciona a próxima ocorrência da palavra (renomear na mão, com cuidado)
<code>F2</code>	renomear símbolo no projeto inteiro (lembrar: campo renomeado perde o valor do Inspector — 1.3)
<code>Ctrl + Shift + O</code>	pular pra um método do arquivo
<code>Ctrl + G</code>	ir pra linha N (a linha do erro do Console)
<code>Ctrl + K, Ctrl + C / Ctrl + K, Ctrl + U</code>	comentar / descomentar bloco

9.4 Editor da Unity — atalhos e hábitos

* Simples assim

O hábito que mais evita dor de cabeça: *Play Mode tint* (o editor fica colorido enquanto roda, pra eu não editar achando que salvou) e *Clear on Play* no Console. E `Ctrl + S` salva só a cena; **File** → **Save Project** salva tags, layers e Input Manager.

atalho / hábito	faz
<code>Ctrl + P</code>	Play / Stop
<code>Ctrl + Shift + P</code>	Pause (e <code>Ctrl + Alt + P</code> avança um quadro — ver colisão quadro a quadro)

atalho / hábito	faz
Ctrl + S	salva só a cena
File → Save Project	salva tags, layers, Input Manager, prefabs — fazer antes de commitar
Ctrl + D	duplica o objeto selecionado
F (na Scene)	enquadra o objeto selecionado
Q W E R T	ferramentas: mão, mover, girar, escalar, retângulo
cadeado do Inspector (ícone no canto)	trava o Inspector num objeto enquanto arrasto outro pra caixinha
Play Mode tint (Edit → Preferences → Colors → Playmode tint)	pintar o editor de cor em Play pra NUNCA editar achando que vai salvar
Console → Clear on Play + Collapse	ligar os dois
Console → Error Pause	pausa no primeiro erro (pegar NullReference no ato)
Inspector → Debug mode (: no canto → Debug)	mostra os campos private (ver estaNoChao mudando)
Aba Game → Maximize On Play desligado	ver Hierarchy/Inspector enquanto roda
Project → Assets/Scripts, Assets/Prefabs, Assets/Sprites	pastas por tipo desde o começo

9.5 Git com Unity — o que viaja e o que não viaja

* Simples assim

O projeto da Unity tem pastas que ela regenera sozinha (Library/) — não vão pro git. E os arquivos .meta são o crachá de cada asset — vão SEMPRE e nunca são apagados. Renomear arquivo dentro da Unity, não no Explorer.

- **.gitignore de Unity (obrigatório):** ignorar Library/, Temp/, Logs/, obj/, UserSettings/, .vs/, .vscode/ (se não quiser levar), *.csproj, *.sln. O modelo oficial fica em github.com/github/gitignore → Unity.gitignore. A Library/ é gigante e a Unity regenera sozinha.
- **.meta files: NUNCA apagar, SEMPRE commitar.** Cada asset tem um .meta com um ID; é ele que liga o script ao componente e o prefab às caixinhas. Apagou o .meta = "The referenced script on this Behaviour is missing" e caixinhas vazias no outro PC.
- **Antes de commitar:** File → Save Project (senão Input Manager/tags ficam só na memória).
- **Renomear/mover arquivo:** DENTRO da Unity (aba Project), não no Explorer — a Unity move o .meta junto.
- **O que não viaja no git:** configurações do VS Code (settings.json de usuário, o commit character do 9.1), UserSettings/ (layout das janelas), extensões. Refazer em cada PC.
- **Sprites e sons grandes:** git normal aguenta um projeto de aula; projeto grande usa Git LFS.

APÊNDICES

A. Cheat sheet — as linhas que mais uso

* Simples assim

As linhas que aparecem em todo script, pra copiar sem pensar. Se uma delas não fizer sentido, o comentário ao lado diz de que peça ela é.

```
// ---- input ----
float lado = Input.GetAxisRaw("Horizontal");           // -1 0 +1
if (Input.GetButtonDown("Jump")) {}                    // apertou agora
if (Input.GetButtonDown("Fire1")) {}                  // segurando
if (Input.GetButtonUp("Jump")) {}                     // soltou agora
if (Input.GetKeyDown(KeyCode.S)) {}                   // tecla crua

// ---- física: as 3 irmãs ----
rb.linearVelocity = new Vector2(velocidade * lado, rb.linearVelocity.y); // andar
rb.linearVelocity = new Vector2(rb.linearVelocity.x, forcaPulo);          // pular
rb.linearVelocity = new Vector2(rb.linearVelocity.x, rb.linearVelocity.y * 0.5f); // cortar
rb.AddForce(direcao * forca, ForceMode2D.Impulse);                       // empurrão
rb.gravityScale = 0f;

// ---- ler a física ----
bool subindo = rb.linearVelocity.y > 0f;
bool noChao = Physics2D.OverlapCircle(checadorDeChao.position, raioChecador, camadaChao); // 3.5

// ---- transform ----
transform.localScale = new Vector3(direcao * escalaBase, escalaY, 1f); // virar / agachar
transform.position = new Vector2(x, y);                                // teleporte (sem física)
transform.Rotate(0f, 0f, 90f * Time.deltaTime);                       // girar

// ---- nascer e morrer ----
GameObject copia = Instantiate(prefab, ponto.position, Quaternion.identity);
copia.GetComponent<Espada>().direcao = direcao;
Destroy(gameObject, 0.15f);

// ---- achar ----
rb = GetComponent<Rigidbody2D>(); // em mim
if (outro.TryGetComponent(out Inimigo i)) i.TomarDano(1); // no outro
Transform player = GameObject.FindWithTag("Player").transform;

// ---- alarmes ----
void OnCollisionEnter2D(Collision2D colisao) { if (colisao.gameObject.CompareTag("Ground")) {} }
void OnTriggerEnter2D(Collider2D outro) { if (outro.CompareTag("Inimigo")) {} }

// ---- tempo ----
timer -= Time.deltaTime; if (timer <= 0f) {}
StartCoroutine(Piscar()); IEnumerator Piscar() { yield return new WaitForSeconds(0.1f); }
Invoke(nameof(Reiniciar), 2f);

// ---- outros ----
Debug.Log($"vel: {rb.linearVelocity}");
SceneManager.LoadScene(SceneManager.GetActiveScene().name);
animator.SetBool("IsJump", !noChao && rb.linearVelocity.y > 0.1f); // 7.6
animator.SetBool("IsFall", !noChao && rb.linearVelocity.y < -0.1f);
animator.SetBool("andando", lado != 0f);
som.PlayOneShot(clip);
```

B. Template de script novo

* Simples assim

Todo script novo começa igual: `using UnityEngine;`, a classe com o nome do arquivo, caixinhas em cima, gavetas embaixo, `Awake` pra se arrumar e `Update` pra trabalhar. Copio este esqueleto e apago o que não usar.

```
using UnityEngine;

public class NomeDoScript : MonoBehaviour // = nome do arquivo .cs
{
    [Header("Ajustes")]
    public float exemplo = 1f;

    [Header("Referências")]
    public Rigidbody2D rb;

    private bool estadoInterno;

    void Awake()
    {
        if (rb == null) rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
    }

    void OnCollisionEnter2D(Collision2D colisao)
    {
    }
}
```

C. Receita de objeto novo (o que pendurar em quê)

* Simples assim

O que cada objeto da cena precisa ter pendurado pra funcionar com os scripts deste manual — uma linha por objeto. Objeto "não funciona"? Conferir a linha dele aqui primeiro.

objeto	componentes	tag / layer
Boneco (player)	Sprite Renderer (sprite com pivô Bottom — 4.6), Rigidbody2D (Dynamic, Gravity 3, Freeze Rotation Z, Interpolate), BoxCollider2D (ou Capsule) com material <code>SemAtrito</code> (3.1), Animator (bools <code>IsJump</code> , <code>IsFall</code> — 7.6), Movimento, Ataque, VidaPlayer, AudioSource; filhos vazios <code>ChecadorDeChao</code> (sola — 3.5), <code>PontoDoGolpe</code> , <code>Cabeca</code>	Player / Player
Chão / plataforma	Sprite Renderer, BoxCollider2D (sem rb)	Ground / Chao
Plataforma móvel	Sprite Renderer, Rigidbody2D Kinematic , BoxCollider2D, script que usa <code>rb.MovePosition</code>	Ground / Chao
Inimigo	Sprite Renderer, Rigidbody2D (Dynamic, Freeze Rotation Z), BoxCollider2D, Inimigo; filho <code>Pe</code>	Inimigo / Inimigo

objeto	componentes	tag / layer
Golpe (prefab)	BoxCollider2D Is Trigger , Rigidbody2D Kinematic , Espada, (Sprite Renderer opcional)	Golpe / Golpe
Moeda (prefab)	Sprite Renderer, CircleCollider2D Is Trigger , Moeda	Moeda / Default
Zona de morte	BoxCollider2D Is Trigger largo, ZonaDeMorte	Morte / Default
Main Camera	Camera, AudioListener, Camera2D	MainCamera
GameManager	GameManager (vazio, sem sprite)	GameController
Canvas/HUD	Canvas, Canvas Scaler, textos TMP, HUD	—

Matriz de colisão (Physics 2D) pra essa receita: desmarcar `Golpe × Player`, `Inimigo × Inimigo`, `Golpe × Golpe`.

D. Glossário rápido

* Simples assim

As palavras do manual, uma frase cada. Quando uma explicação usar um termo que eu não lembro, é aqui.

termo	significado no manual
componente	qualquer coisa pendurada num GameObject (Rigidbody2D, Collider, meu script)
GameObject	o objeto da cena: um nome, um Transform e componentes
prefab	GameObject salvo como molde na pasta Assets (azul)
Inspector	painel que mostra as caixinhas do objeto selecionado
Hierarchy	lista dos objetos da cena, com pai/filho
cena	um arquivo <code>.unity</code> : uma fase, um menu
campo	variável no topo da classe (caixinha ou gaveta)
propriedade	caixa com regras por trás (<code>linearVelocity</code> , <code>localScale</code>) — se atribui, não se chama
método	bloco com nome que roda quando chamado (meu) ou quando a Unity chama (alarme)
parâmetro / argumento	o que o método recebe (declarado) / o que eu passo (na chamada)
struct	tipo que se carimba inteiro (<code>Vector2</code> , <code>Vector3</code>) — não dá pra mudar só o <code>.y</code>
null	caixa de objeto vazia
quadro (frame)	uma passada do <code>Update</code> ; ~60 por segundo
passo da física	uma passada do <code>FixedUpdate</code> ; 50 por segundo
layer	camada numerada pra decidir quem colide com quem e filtrar raios

termo	significado no manual
tag	etiqueta de texto pra reconhecer objetos no código
trigger	collider fantasma: não empurra, só avisa
corrotina	método que pausa com <code>yield</code> e continua depois
enum	lista fixa de nomes (estados, tipos)
static	pertence à classe, não ao objeto; existe uma vez só
singleton	o padrão <code>Instance</code> : um objeto único acessível de qualquer lugar
stub	a <code>UnityEngine</code> de mentira do PC do trabalho — só compila, não roda
pivô	o ponto do sprite que fica parado quando o objeto escala ou gira; <i>Bottom</i> = a sola (4.6)
LayerMask	caixinha de checkboxes que diz quais layers uma pergunta física (círculo, raio) enxerga (3.3, 3.5)
checador de chão	objeto vazio filho, na sola, onde o <code>OverlapCircle</code> pergunta se tem chão (3.5)
gizmo	desenho de apoio só na aba Scene (o círculo amarelo) — não aparece no jogo (3.5)
Physics Material 2D	asset com atrito e quique; <code>Friction 0</code> = o boneco não gruda na parede (3.1)
sombra (shadowing)	variável local com o mesmo nome de um campo: a nova esconde a do topo dentro do método (1.11)
parâmetro do Animator	interruptor (<code>bool</code>), número (<code>float</code>) ou gatilho (<code>trigger</code>) que o script liga e as setas do Animator leem (7.6)

Fim desta edição (04/09/2026). Peça nova nas aulas → entrada nova aqui, no mesmo formato: o que é → pra que serve → regras → onde tá no meu código → exemplos, e a caixa Simples assim pra quem chega depois.